A SOURCE-BASED LEARNING GUIDE

rc_handle_cpp

C++ 与 ROS 2 初学者源码详解教程

从编译和启动，到读懂每个函数与构建规则

C++17 | ROS 2 Humble | Ubuntu 22.04

重点内容

源码逐段讲解与原始行号

package.xml：身份、依赖与构建类型

CMakeLists.txt：目标、链接、安装与导出

launch / YAML / 消息 / 测试 / 动手练习

依据当前项目实际源码制作 · 17 章 · 可点击目录与书签

源码快照 2026-10-24 | 包含编译与自动测试核验说明

工作空间 /home/fyp/rc_handle

目录与阅读路线

点击章名跳转；PDF 书签中还可展开各节。首次阅读可先看第 2、3、7、8、11 章，再深入构建规则与设备层。

01 阅读指南：从能运行到能读懂 3

02 先认识整个项目 5

03 编译、启动和观察数据 7

04 逐行读懂 package.xml 9

05 逐段读懂 CMakeLists.txt 12

06 读源码需要的 C++ 和消息基础 17

07 joy_node_main.cpp：程序从哪里开始 22

08 JoyNode：参数、定时器和消息发布 24

09 JoystickDevice：Linux 手柄读取的底层 29

10 joy_raw_listener.cpp：订阅者怎样写 34

11 launch 和 YAML：把配置送到节点 36

12 把运行过程连成一条时间线 39

13 阅读测试：没有硬件怎样验证行为 41

14 动手练习：每一步都能检查结果 44

15 常见问题与定位方法 47

16 速查表与自测题 49

17 参考资料、版本和源码清单 52

01 阅读指南：从能运行到能读懂

本教程面向同时刚开始学习 C++ 和 ROS 2 的读者，以工作空间 `/home/fyp/rc_handle` 中的实际代码为例。阅读目标是：独立编译和启动 C++ 手柄驱动，追踪一条手柄消息的产生过程，读懂每个源文件的职责，并能有依据地修改 `package.xml`、`CMakeLists.txt`、launch 和 YAML 配置。

版本范围：Ubuntu 22.04、ROS 2 Humble、C++17；源码快照日期：2026-10-24。 项目使用 Linux 的 `joydev` 接口和 `/dev/input/js*`，涉及 `<linux/joystick.h>`、`open/read/ioctl/close`。Windows 原生环境不能直接照搬这些设备读写代码；若通过 WSL 或虚拟机学习，仍需让 Linux 环境实际识别手柄设备。

1.1 这本教程怎样使用

第一遍读第 2、3、7、8、11 章，先建立“运行程序—节点—话题—回调”的联系。第二遍重点读第 4、5 章，理解包清单和编译规则。第三遍读第 9、12、13 章，分析 Linux 设备读取、异常恢复和自动测试。第 14、15 章用于练习和排错。

文中“源码 L12”表示对应文件的第 12 行。源码清单由当前文件自动摘录，包含原始行号；带有“教学示例”的代码是便于理解的简化写法，不能直接当作已修改的项目源码。命令中的工作空间绝对路径按本机填写，换电脑时应替换成你自己的位置。

文件类型	在项目中的用途	阅读时要回答的问题
`.hpp`	声明类、成员、函数接口	这个对象保存什么，能调用什么？
`.cpp`	定义函数和程序入口	每一步实际做什么，失败后怎么办？
`package.xml`	声明包身份、依赖和构建类型	ROS 工具怎样识别包和依赖关系？
`CMakeLists.txt`	描述目标、编译、链接、安装和导出	哪些文件变成哪个程序或库？
`.launch.py`	组织节点启动和参数传递	启动哪个包、哪个程序、使用哪些话题？
`.yaml`	保存节点参数	设备、频率、重连间隔从哪里来？
`.msg`	定义通信消息字段	发布者和订阅者交换什么数据？

1.2 先记住三个约定

1. C++ 包是 `rc_handle_cpp`，Python 包是 `rc_handle`。两者都有 `joy_node` 入口，启动命令中的包名决定实现。

2. 两个话题都保留原始量级：`/joy_raw` 用 int16/uint8，`/joy` 用 float32/int32。原值 `-32768` 只变成 `-32768.0`，没有归一化、软件死区或轴反转。

3. 读取故障时各话题发一次全零状态，再等待重连；启动时没有设备则不发周期数据。参数只读，修改 YAML 后需要重新启动节点。

1.3 本次文档制作中的核验范围

本次核对了核心源码与配置，执行了构建、launch 参数展示、消息接口查询和已有 Google Test，详情见第 13 章。本次环境没有 `/dev/input/js0`，未进行真实手柄实测。

02 先认识整个项目

2.1 工作空间、包、节点和程序

工作空间是组织和编译一组 ROS 包的目录。这里的根目录是 `/home/fyp/rc_handle`。其 `src/` 放源码，`build/` 放中间文件，`install/` 放运行和被其他包使用的结果，`log/` 放构建或测试日志。

包是具有 `package.xml` 的组织单位。节点是运行时参加 ROS 通信的对象；一个进程中可以有多个节点。可执行程序是操作系统可以启动的文件。这里 `joy_node_main.cpp` 的 `main()` 创建一个 `JoyNode`，因此通常一个 `joy_node` 进程里运行一个驱动节点。

结构与输出示意

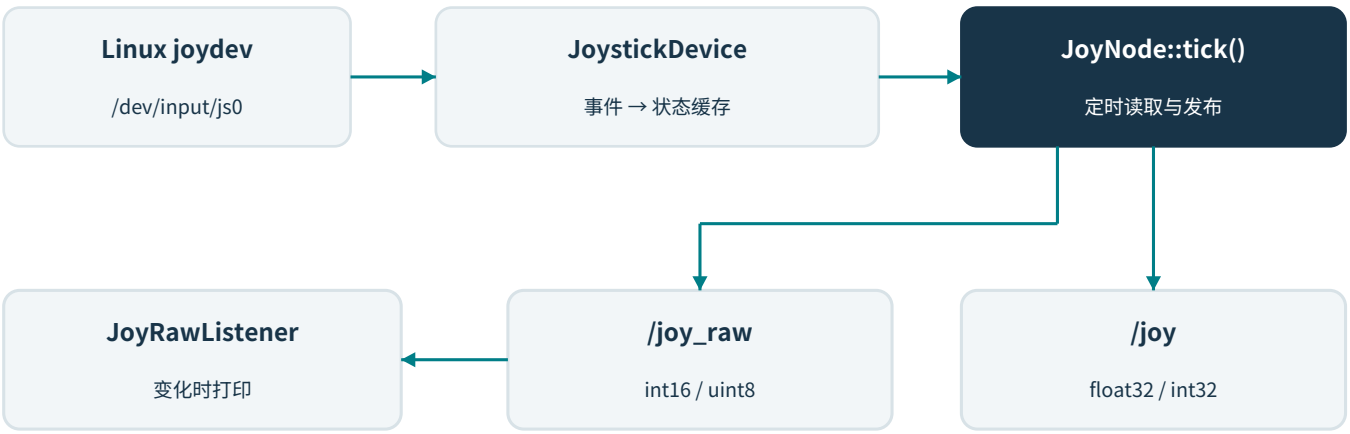
```
/home/fyp/rc_handle/
  src/
    rc_handle/                Python implementation
    rc_handle_cpp/            C++ implementation
      package.xml
      CMakeLists.txt
      include/rc_handle_cpp/
        joystick_device.hpp
        joy_node.hpp
      src/
        joystick_device.cpp
        joy_node.cpp
        joy_node_main.cpp
        joy_raw_listener.cpp
      launch/joy.launch.py
      config/joy.yaml
      test/
        test_joystick_device.cpp
        test_joy_node.cpp
    rc_handle_msgs/           Shared message definitions
      msg/JoyRaw.msg
      package.xml
      CMakeLists.txt
  build/
  install/
  log/
```

名称	本项目实际值	出现在哪里
包名	`rc_handle_cpp`	XML、CMake、`ros2 run`、launch 的 `package`
发布程序名	`joy_node`	`add_executable`、`ros2 run`、launch 的 `executable`
默认发布节点名	`rc_handle_cpp_node`	`JoyNode` 构造函数、launch 的 `name`
订阅程序名	`joy_raw_listener`	CMake 和 `ros2 run`
订阅节点名	`joy_raw_listener_cpp`	订阅类的基类构造函数
默认原始话题	`/joy_raw`	创建发布器、订阅器；根命名空间下解析
默认兼容话题	`/joy`	`sensor_msgs::msg::Joy` 发布器

名称	本项目实际值	出现在哪里
CMake 驱动库目标	`rc_handle_cpp_driver`	两个核心 `.cpp` 组成的共享库

源码中的 `"joy_raw"` 是相对话题名。在默认根命名空间下，它解析为 `/joy_raw`；如果设置命名空间 `/robot1`，它通常解析为 `/robot1/joy_raw`。前面带 `/` 的名字是绝对话题名。

2.2 一次输入怎样到达订阅者



同一次发布使用相同 header；两个话题均保留原始数值量级。

手柄发生变化，Linux 内核产生 `js_event`。`JoystickDevice::read_state()` 读取事件，交给 `JoystickState::apply()` 更新缓存。`JoyNode::tick()` 在定时器回调中调用读取，再通过 `publish_state()` 发出两个 ROS 消息。`JoyRawListener` 收到消息后，仅在轴或按键发生变化时打印。

这里既有“事件”，也有“状态”。设备产生的事件描述某个轴或按键的变化；驱动发出的消息描述所有轴和按键的当前完整状态。相同状态可以被重复发布，不能用日志行数直接计算话题频率。

2.3 为什么拆成多个文件

`joystick_device.*` 处理设备协议和文件描述符；`joy_node.*` 处理 ROS 参数、定时器和发布器；`joy_node_main.cpp` 处理进程启动；`joy_raw_listener.cpp` 演示订阅。这样设备处理逻辑可以独立测试，节点也可以注入模拟设备，而不必每次测试都插入真实手柄。

头文件放在 `include/rc_handle_cpp/`，源文件中写 `#include "rc_handle_cpp/joy_node.hpp"`。CMake 添加的是外层 `include/` 搜索路径，因此编译器能够继续找到内部的 `rc_handle_cpp/joy_node.hpp`。

03 编译、启动和观察数据

3.1 准备终端环境

以下命令以已经安装 ROS 2 Humble 的 Ubuntu 为前提。先加载 ROS 基础环境，再进入工作空间：

终端命令

```
source /opt/ros/humble/setup.bash
cd /home/fyp/rc_handle
colcon list
```

`source` 让环境脚本在当前 shell 中生效。它设置包搜索路径、库路径、Python 路径等。每个终端有自己的环境；在终端 A 加载环境，不会自动让终端 B 也加载。

已有 ROS 安装但缺少依赖时，可以在工作空间根目录使用以下命令。`rosdep install` 会实际安装依赖，通常需要网络和系统安装权限；它不负责把你的源码编译成节点。

终端命令

```
rosdep install --from-paths src --ignore-src -r -y --rosdistro humble
```

3.2 编译 C++ 包及消息依赖

终端命令

```
colcon build --symlink-install --packages-up-to rc_handle_cpp
source install/setup.bash
ros2 pkg executables rc_handle_cpp
```

预期至少列出两个入口：

结构与输出示意

```
rc_handle_cpp joy_node
rc_handle_cpp joy_raw_listener
```

`--packages-up-to rc_handle_cpp` 会选择目标包及工作空间中的相关依赖，包含 `rc_handle_msgs`。`--packages-select rc_handle_cpp` 只选择这个包，适合依赖已经构建好的场景。初次构建使用前者更容易避免“找不到消息包”。

`--symlink-install` 在可支持的安装项上使用符号链接，有利于开发。C++ 源码修改后仍需要重新编译；修改安装规则、增加目标或增加安装文件后，也应重新构建。不要把它理解为“修改任何文件都立刻生效”。

3.3 终端 A：运行发布节点

终端命令

```
source /opt/ros/humble/setup.bash
source /home/fyp/rc_handle/install/setup.bash
ros2 launch rc_handle_cpp joy.launch.py
```

连接成功时，日志包含设备名和轴、按键数量。若设备不存在，日志会报告 `Cannot open`，节点保持运行并按间隔重试。同一条打开错误会被去重，日志没有每秒重复打印，并不表示重连停止了。

也可以直接运行可执行程序：

终端命令

```
ros2 run rc_handle_cpp joy_node --ros-args \
  -p dev:=/dev/input/js0 \
  -p publish_rate:=50.0 \
  -p reconnect_interval:=1.0
```

这两个命令应选择一个运行。学习当前流程时，也不要同时启动 Python 和 C++ 发布节点向同一话题发送手柄状态。

3.4 终端 B：运行订阅示例

终端命令

```
source /opt/ros/humble/setup.bash
source /home/fyp/rc_handle/install/setup.bash
ros2 run rc_handle_cpp joy_raw_listener
```

预期日志形如下面的示意数据；实际数量和值由你的手柄决定：

结构与输出示意

```
axes=[0, -1234, 32767] buttons=[0, 1, 0]
```

3.5 终端 C：查询 ROS 图和消息

终端命令

```
source /opt/ros/humble/setup.bash
source /home/fyp/rc_handle/install/setup.bash
ros2 node list
ros2 node info /rc_handle_cpp_node
ros2 topic list -t
ros2 topic info /joy_raw --verbose
ros2 interface show rc_handle_msgs/msg/JoyRaw
ros2 topic echo /joy_raw rc_handle_msgs/msg/JoyRaw
```

`topic echo` 持续订阅，按 `Ctrl+C` 停止。要测频率，可在另一个终端运行 `ros2 topic hz /joy_raw`。默认目标频率是 50 Hz，实际观察结果受调度和系统负载影响。没有成功打开设备时不会有周期数据，`hz` 也不能显示正常发布频率。

3.6 没有手柄也可以学习哪些部分

构建、消息接口查看、`ros2 launch ... --show-args` 和已有自动测试都可以在没有真实手柄时学习。测试通过模拟设备验证节点行为。真实的 `/dev/input/js0` 读取、硬件按键映射和实际断线恢复，则需要可用的 Linux 手柄设备。

04 逐行读懂 package.xml

`package.xml` 是 ROS 包的清单，供包管理、依赖分析和构建工具识别。本项目采用格式 3；`format="3"` 是清单格式版本，不表示 ROS 3。其标签定义可对照 REP-149：包清单格式 3。

4.1 当前文件完整清单

源码：`src/rc_handle_cpp/package.xml`，L1—L21。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/package.xml

```xml
<?xml version="1.0"?>
<package format="3">
  <name>rc_handle_cpp</name>
  <version>0.1.0</version>
  <description>C++ Linux joystick driver preserving int16 axes and uint8 buttons.</description>
  <maintainer email="fyp@example.com">fyp</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>
  <depend>rcl_interfaces</depend>
  <depend>sensor_msgs</depend>
  <depend>rc_handle_msgs</depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <test_depend>ament_cmake_gtest</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

4.2 L1—L7：包的身份信息

`<?xml version="1.0"?>` 声明 XML 文档版本。XML 使用成对的开始和结束标签，例如 `<name>rc_handle_cpp</name>`。`email="..."` 是标签属性。`<package>` 是外层根元素，必须正常闭合。

`<name>` 是 ROS 包名，与 CMake 中 `project(rc_handle_cpp)` 一致。版本号 `0.1.0` 用于记录包自身版本。`<description>` 是功能描述。`<maintainer>` 指维护者和邮箱；这里是项目中的示例联系方式，正式发布时应替换成实际信息。`<license>` 声明许可证名称，项目同时安装 `LICENSE` 文件。

包名、节点名、可执行程序名是三个不同层次：修改 `<name>` 不会自动修改类名、可执行目标名或 launch 配置。修改包名时必须同步检查所有引用。

4.3 L9：构建工具依赖

XML 片段

```xml
<buildtool_depend>ament_cmake</buildtool_depend>
```

它声明这个包在构建过程中使用 `ament_cmake`。`ament_cmake` 为 ROS 包提供 CMake 宏和安装导出规则；具体使用这些能力的地方仍然在 `CMakeLists.txt` 中。

4.4 L10—L13：编译和使用需要的依赖

<depend> 是常用的综合依赖声明。在格式 3 中，它覆盖 build_depend、build_export_depend 和 exec_depend 所表达的相应依赖需求；构建工具依赖和测试依赖仍有各自标签。这里的用途如下：

依赖	当前代码中使用的位置	作用
rclcpp	两个头文件和节点入口、订阅程序	ROS 2 C++ 节点、发布订阅、参数、执行器和日志
rcl_interfaces	joy_node.cpp 中的 ParameterDescriptor	设置参数的只读属性
sensor_msgs	sensor_msgs::msg::Joy	标准手柄消息类型
rc_handle_msgs	rc_handle_msgs::msg::JoyRaw	本工作空间的原始整数消息类型

rclcpp 不是 C++ 语言本身，而是 ROS 2 的 C++ 客户端库。rc_handle_msgs 也不是一个外部手柄驱动，它只负责接口定义和生成相关类型。

依赖声明会帮助 colcon 计算包的构建顺序，也可供 rosdep 解析安装需求。单独写一个 <depend> 不会自动执行系统安装，更不会自动把某个库链接到你的可执行目标。

4.5 L14—L16：运行和测试依赖

XML 片段

```xml
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
<test_depend>ament_cmake_gtest</test_depend>
```

launch 提供启动描述和动作，launch_ros 提供启动 ROS 节点等扩展。本项目的驱动逻辑用 C++ 编写，但 .launch.py 仍需要这些 Python 启动组件。

ament_cmake_gtest 用来配置 Google Test 测试目标。只有在构建测试部分时才需要它，本项目通过 if(BUILD_TESTING) 包围相应构建规则。

4.6 L18—L20：向工具导出构建类型

XML 片段

```xml
<export>
  <build_type>ament_cmake</build_type>
</export>
```

这告诉构建工具按照 ament_cmake 类型处理该包。<export> 中的构建类型与前面的构建工具依赖相互配合。

4.7 package.xml 与 CMakeLists.txt 怎样配合

你的需求	package.xml 中处理什么	CMake 中处理什么
使用 `JoyRaw`	声明依赖 `rc_handle_msgs`	查找该包，并把依赖应用到使用它的目标
使用 rclcpp 创建节点	声明依赖 `rclcpp`	`find_package` 与 `ament_target_dependencies`
安装 launch	声明运行需要 `launch`、`launch_ros`	`install(DIRECTORY launch ...)`
增加 GTest	声明测试依赖	在测试条件中创建并链接测试目标

把 XML 想成“包需要什么”的清单，把 CMake 想成“这些东西如何参与构建”的规则。遇到新增依赖，两处通常都要检查。

4.8 常见错误与自查

标签漏写结束符会导致 XML 解析失败；把 `rclcpp` 拼成别的名字会使依赖查找失败；`<name>` 与 `project()` 不一致会破坏包配置。`package.xml` 文件名固定，当前文件也不是一个 XML 格式的 launch 文件。

教学自查：你能否指出为什么这里有 `launch_ros` 运行依赖，但 CMake 没有把它链接到 `joy_node`？答案是它服务于启动脚本，而 C++ 程序链接的是自身使用的 C++ 依赖。

05 逐段读懂 CMakeLists.txt

文件的准确名称是 `CMakeLists.txt`，注意 `Lists` 的拼写和大小写。CMake 在配置阶段读取它，生成构建规则；编译器根据规则编译 C++，链接器再把目标文件和库组成最终程序。`colcon` 负责协调多个 ROS 包的这一过程。

ROS 专用宏可对照 ament_cmake 的 Humble 官方说明。下面的目标名、安装路径和行号全部来自当前项目。

5.1 当前文件完整清单

源码：`src/rc_handle_cpp/CMakeLists.txt`，L1—L57。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(rc_handle_cpp)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rcl_interfaces REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(rc_handle_msgs REQUIRED)

add_library(${PROJECT_NAME}_driver SHARED
  src/joystick_device.cpp
  src/joy_node.cpp
)
target_include_directories(${PROJECT_NAME}_driver PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>
)
ament_target_dependencies(${PROJECT_NAME}_driver
  rclcpp rcl_interfaces sensor_msgs rc_handle_msgs)

add_executable(joy_node src/joy_node_main.cpp)
target_link_libraries(joy_node ${PROJECT_NAME}_driver)
ament_target_dependencies(joy_node rclcpp)

add_executable(joy_raw_listener src/joy_raw_listener.cpp)
ament_target_dependencies(joy_raw_listener rclcpp rc_handle_msgs)

install(TARGETS ${PROJECT_NAME}_driver
  EXPORT export_${PROJECT_NAME}_driver
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION bin
)
install(TARGETS joy_node joy_raw_listener DESTINATION lib/${PROJECT_NAME})
install(DIRECTORY include/ DESTINATION include)
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})
install(FILES README.md LICENSE DESTINATION share/${PROJECT_NAME})

if(BUILD_TESTING)
  find_package(ament_cmake_gtest REQUIRED)
  ament_add_gtest(test_joystick_device test/test_joystick_device.cpp)
  target_link_libraries(test_joystick_device ${PROJECT_NAME}_driver)
  ament_add_gtest(test_joy_node test/test_joy_node.cpp TIMEOUT 30)
```

src/rc_handle_cpp/CMakeLists.txt（续）

```
51    target_link_libraries(test_joy_node ${PROJECT_NAME}_driver)
52    ament_target_dependencies(test_joy_node rclcpp rc_handle_msgs sensor_msgs)
53  endif()
54
55  ament_export_targets(export_${PROJECT_NAME}_driver HAS_LIBRARY_TARGET)
56  ament_export_dependencies(rclcpp rcl_interfaces sensor_msgs rc_handle_msgs)
57  ament_package()
```

5.2 L1—L9：工程和编译选项

`cmake_minimum_required(VERSION 3.8)` 指定最低 CMake 版本；`project(rc_handle_cpp)` 设置工程名称，并使 `${PROJECT_NAME}` 展开为 `rc_handle_cpp`。`${...}` 是 CMake 变量引用语法，CMake 命令的参数通常由空格或换行分隔。

`CMAKE_CXX_STANDARD 17` 选择 C++17；`CMAKE_CXX_STANDARD_REQUIRED ON` 要求满足相应标准；`CMAKE_CXX_EXTENSIONS OFF` 关闭依赖编译器扩展的标准模式。`if(... MATCHES "GNU|Clang")` 为 GCC 或 Clang 添加编译警告。`-Wall -Wextra -Wpedantic` 帮助发现可疑代码，但并不自动把所有警告当成错误。

5.3 L11—L15：查找依赖包

CMake 片段

```
1  find_package(rclcpp REQUIRED)
```

这要求 CMake 找到 `rclcpp` 的包配置。`REQUIRED` 表示找不到时停止配置。环境中的安装前缀和包提供的配置文件会影响查找结果。因此“已经安装但忘了 source”与“根本没有安装”都可能表现为查找失败。

`find_package` 解决发现依赖的问题；具体目标的头文件路径、链接需求等，还要通过后续目标设置建立联系。本项目对 ROS 依赖使用 `ament_target_dependencies`。

5.4 L17—L20：把核心逻辑编译成共享库

CMake 片段

```
1  add_library(${PROJECT_NAME}_driver SHARED
2    src/joystick_device.cpp
3    src/joy_node.cpp
4  )
```

展开后的目标名是 `rc_handle_cpp_driver`。`SHARED` 表示共享库，Linux 下通常形成 `librc_handle_cpp_driver.so`。这个库包含设备处理和节点类的实现，不包含 `main()`，所以它本身不是你通过 `ros2 run` 启动的程序。

这里不需要把每个 `.hpp` 都列进 `add_library`。编译器通过源文件的 `#include` 找到声明。头文件是否能被找到，由头文件搜索路径控制。

5.5 L21—L24：头文件路径和 PUBLIC

CMake 片段

```
target_include_directories(${PROJECT_NAME}_driver PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>
)
```

`target_include_directories` 为特定目标配置包含目录。`PUBLIC` 表示本目标编译要使用，链接并使用该目标的下游也要得到相应使用要求。对比之下，`PRIVATE` 只给自身使用，`INTERFACE` 只作为下游使用要求。

`$<...>` 是 CMake 生成器表达式。构建当前源码时，`BUILD_INTERFACE` 使用包源码目录内的 `include/`；安装后被其他包引用时，`INSTALL_INTERFACE:include` 指向安装前缀中的 `include/`。这样下游无需依赖你电脑上的源码绝对路径。

例如编译器把 `include/` 加到搜索路径后，就可以找到 `rc_handle_cpp/joy_node.hpp`。不要把搜索目录错误地设成 `include/rc_handle_cpp` 后仍写同样的 include 路径，否则会多出一层。

5.6 L25—L26：把 ROS 依赖应用到库目标

CMake 片段

```
ament_target_dependencies(${PROJECT_NAME}_driver
  rclcpp rcl_interfaces sensor_msgs rc_handle_msgs)
```

该宏为这个驱动库配置声明的依赖所需的包含目录、库及相关使用要求。注意依赖加在实际使用它们的目标上：驱动库包含 `JoyNode` 实现，所以它需要 `rclcpp` 和消息包；订阅示例也使用 `JoyRaw`，所以它有自己的依赖设置。

5.7 L28—L30：生成发布可执行程序

CMake 片段

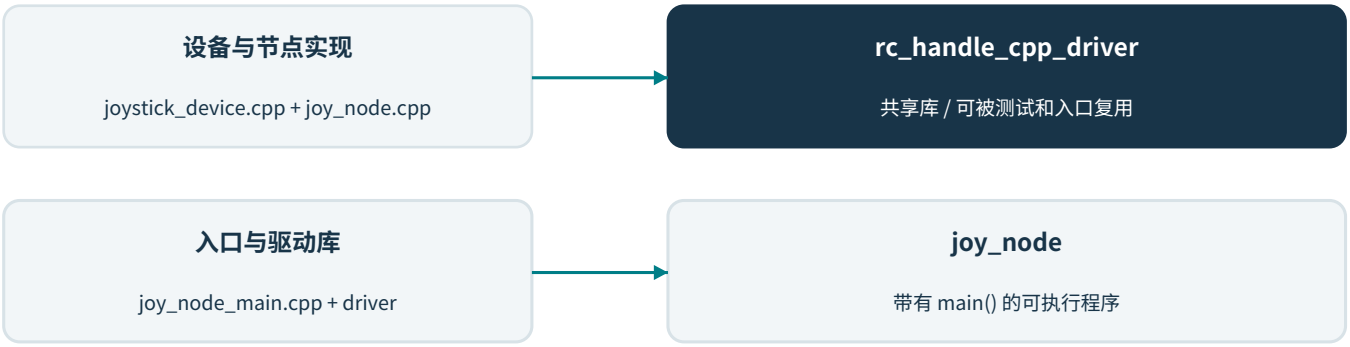
```
add_executable(joy_node src/joy_node_main.cpp)
target_link_libraries(joy_node ${PROJECT_NAME}_driver)
ament_target_dependencies(joy_node rclcpp)
```

第一句创建名为 `joy_node` 的可执行目标，入口来自 `joy_node_main.cpp`。第二句把它和驱动库连接起来，因此 `main()` 才能调用库里的 `JoyNode` 构造函数等实现。第三句处理入口文件直接使用的 `rclcpp` 依赖。

`#include` 让编译器看见声明；链接库让链接器找到对应实现。如果只 include 头文件却没链接实现，常见错误是 `undefined reference`。如果忘记 include 搜索路径，常见错误是 `No such file or directory`，两类问题发生在不同阶段。

5.8 L32—L33：生成订阅可执行程序

`joy_raw_listener.cpp` 自己包含 `main()` 和订阅类，所以可以直接建立第二个可执行目标。它没有使用 `JoystickDevice` 或 `JoyNode` 类，因此没有链接本包的驱动库，但仍需要 `rclcpp` 和 `rc_handle_msgs`。



joy_raw_listener.cpp 独立生成订阅程序；测试文件链接驱动库。

CMake 负责建立这些目标关系，再通过 install() 安装可运行结果。

5.9 L35—L44：安装目标和资源

构建目录里的程序不等于已经注册成 ROS 包可查找的运行入口。本项目把两个程序安装到 `lib/${PROJECT_NAME}`，这是 `ros2 run` 在包前缀下寻找可执行入口所使用的布局。

CMake 安装项	默认隔离安装布局示例	用途
共享库的 `LIBRARY DESTINATION lib`	`install/rc_handle_cpp/lib/librc_handle_cpp_driver.so`	运行时装载驱动实现
两个可执行目标	`install/rc_handle_cpp/lib/rc_handle_cpp/`	供 `ros2 run` 和 launch 启动
`include/` 目录	`install/rc_handle_cpp/include/rc_handle_cpp/`	下游包引用公开头文件
`launch`、`config`	`install/rc_handle_cpp/share/rc_handle_cpp/`	启动文件和 YAML 参数文件
`README.md`、`LICENSE`	同一 `share/rc_handle_cpp/` 下	使用说明和许可证

表中路径按当前默认的包隔离安装布局举例；采用 `--merge-install` 时前缀布局会不同。`ARCHIVE` 一般处理静态库或导入库，`LIBRARY` 在 Linux 处理共享库，`RUNTIME` 处理可执行等运行目标；不同平台的目标分类可能不同。

`install(DIRECTORY include/ DESTINATION include)` 源目录末尾的 `/` 表示安装其内部内容。`install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})` 则把两个目录作为目录一起安装。

如果 CMake 里写出了 launch 文件的安装规则，但还没有重新构建，`ros2 launch` 仍可能找不到它。ROS 工具通常通过安装后的包索引和 share 目录定位资源。

5.10 L46—L53：创建测试目标

`if(BUILD_TESTING)` 决定是否处理测试规则。`ament_add_gtest` 创建 Google Test 目标并注册测试；随后链接同一个驱动库，保证测试使用实际实现。`test_joy_node` 的 `TIMEOUT 30` 用来避免异常情况下无限等待。

`test_joystick_device` 主要测试状态更新和失败路径；`test_joy_node` 创建节点与模拟设备，验证 ROS 消息发布、断线和重连。这些测试文件不会进入 `joy_node` 的正常运行主循环。

5.11 L55—L57：导出与完成包配置

`install(TARGETS ... EXPORT export_rc_handle_cpp_driver)` 建立安装导出集合，
`ament_export_targets(... HAS_LIBRARY_TARGET)` 将目标信息向下游包导出。
`ament_export_dependencies(...)` 让下游使用该库时获得必要的依赖查找关系。

文件末尾的 `ament_package()` 完成包配置和 ament 索引等安装信息的生成。本文件只调用一次。它应位于这里的导出设置之后，便于收集前面已经声明的信息。

5.12 修改不同文件后要做什么

修改内容	通常需要的操作	原因
`.cpp`、`.hpp`	重新 `colcon build`，再启动新进程	C++ 要重新编译或链接
新增 `.cpp` 可执行程序	加 `add_executable`、依赖、安装规则，再构建	文件放进 src 不会自动变成目标
增加 ROS 依赖	同步 XML、CMake，再构建	元数据和目标规则都要更新
`.msg`	先重建消息包及相关使用者	需要重新生成接口代码
YAML 参数值	使用更新后的参数文件并重启节点	当前节点参数为启动时只读
新增 launch/config 文件	更新必要安装规则，再构建	确保安装目录中能找到资源

06 读源码需要的 C++ 和消息基础

6.1 声明、定义、头文件和命名空间

头文件里的 `void read_state();` 是函数声明，告诉编译器函数名字和调用方式。源文件里的 `void JoystickDevice::read_state() { ... }` 是定义，提供函数体。`JoystickDevice::` 表示这个函数属于该类。

`namespace rc_handle_cpp { ... }` 把项目类型放进自己的命名空间，减少与其他库的同名冲突。使用完整名字时写 `rc_handle_cpp::JoyNode`。代码中的 `std::vector`、`std::string` 来自 C++ 标准库命名空间 `std`。

`.hpp` 的 `#ifndef`、`#define`、`#endif` 是头文件保护，避免一个编译单元反复包含同一声明。`#include <...>` 常用于系统或库头文件，`#include "..."` 常用于项目头文件；两者的具体搜索顺序由编译器及包含路径配置决定。

6.2 类、结构体与成员访问

`JoystickState` 使用 `struct` 保存轴和按键；`JoystickDevice`、`JoyNode` 使用 `class` 封装行为。`struct` 默认成员公开，`class` 默认成员私有；两者都可以拥有数据、成员函数、构造函数和继承关系。

`public` 是调用者可用的接口；`private` 是类内部实现；`protected` 让本类和派生类访问。本项目的模拟设备需要设置 `state_`，所以它在设备基类中是 `protected`。数据成员后缀 `_` 是代码命名习惯，不是 C++ 语法要求。

对象用点访问成员，指针和智能指针常用箭头：

C++ 教学片段

```cpp
JoystickState state;
state.reset(2, 3);
auto device = std::make_unique<JoystickDevice>("/dev/input/js0");
device->open();
```

上面是教学片段，省略了命名空间或相应 `using`。`reset(2, 3)` 表示两个轴、三个按键，不表示第 2 个轴和第 3 个按键。

6.3 构造函数和初始化列表

C++ 教学片段

```cpp
JoystickDevice::JoystickDevice(std::string path)
: path_(std::move(path)), name_(path_)
{
}
```

构造函数与类同名，没有返回类型。冒号后的初始化列表用于初始化成员。真实初始化顺序由成员在类中声明的顺序决定，不由列表书写顺序决定。这里先初始化 `path_`，再把它作为初始设备名使用。

`explicit` 限制不希望发生的隐式类型转换，例如避免一个字符串在不明显的地方自动变成设备对象。析构函数写作 `~JoystickDevice()`，在对象生命周期结束时执行，当前实现用它关闭设备。

6.4 const、引用与 auto

代码写法	在本项目中的含义
`const js_event & event`	通过引用读取原事件，避免复制，并禁止通过此引用修改事件
`const JoystickState & state()`	返回内部状态的只读引用；调用者不取得所有权
`is_open() const`	这个成员函数不通过普通成员访问修改对象状态
`const auto & state = device_->state()`	让编译器推断引用类型，读取缓存而不复制整个向量
`auto bytes = ::read(...)`	从函数返回值推断变量类型，保留正数和负数语义

`&` 在不同位置有不同含义：类型中的 `const T &` 表示引用；表达式中的 `&axis_count` 表示取地址；两个值之间的 `a & b` 表示按位与。读代码时先看它处于什么语法位置。

6.5 vector、array 与固定宽度整数

`std::vector<T>` 是动态长度数组，用于轴和按键数由设备决定的情况。`std::array<js_event, 256>` 是固定长度数组，用于每次批量读取事件的缓冲区。

C++ 教学片段

```cpp
std::vector<std::int16_t> axes;
axes.assign(3, 0);
axes[1] = -1234;
```

执行后 `axes` 为 `[0, -1234, 0]`。索引从 0 开始，`size()` 返回元素数。`operator[]` 不自动检查越界，所以真实源码在写入前检查 `event.number < axes.size()`。

`std::int16_t` 是有符号 16 位整数，可表示 `[-32768, 32767]`；`std::uint8_t` 是无符号 8 位整数，可表示 `[0, 255]`。正常按键多用 0/1，本项目没有强制转换成 bool；这可以在测试中保留 2 和 255 等可表示值。

6.6 智能指针和所有权

`std::unique_ptr<JoystickDevice>` 表示独占拥有设备对象。它不能被普通复制；`std::move(device)` 允许把所有权转移到节点成员。`std::move` 本身是转换表达式，真正的移动由接收方的移动构造或赋值完成。

`std::shared_ptr<JoyNode>` 允许多个持有者共同维持节点生命周期，最后一个持有者释放时才销毁对象。`std::make_shared<JoyNode>()` 创建对象并返回共享智能指针，适合交给 `rclcpp::spin`。

智能指针管理 C++ 对象；Linux 的整数文件描述符仍由 `JoystickDevice::close()` 管理。这两层资源管理共同组成当前驱动的生命周期。

6.7 继承、virtual 与测试替身

`class JoyNode : public rclcpp::Node` 表示 `JoyNode` 继承 ROS 节点能力。设备类将 `open/read_state/close/is_open` 声明为 `virtual`，测试中的 `ScriptedDevice` 可以用 `override` 覆盖它们。

`virtual ~JoystickDevice()` 保证通过基类接口销毁派生设备对象时，正确执行派生类析构过程。复制构造和复制赋值被 `= delete` 禁止，避免两个设备对象误以为自己都拥有同一个文件描述符。

6.8 Lambda、异常和 noexcept

C++ 教学片段

```
1 [this]() {tick();}
```

这是匿名函数，即 Lambda。`[this]` 让函数体可以访问当前对象，`()` 是参数列表，`{tick();}` 是执行内容。这个函数被交给定时器，后续由执行器调用；创建定时器时不会就把所有未来回调执行完。

`throw` 抛出异常，`try/catch` 在适当层捕获。当前设备层抛出 `std::system_error`，节点层捕获可恢复的设备故障并重连；入口层捕获未被处理的 `std::exception`，记录严重错误并返回失败码。`noexcept` 表示函数承诺不向调用者抛出异常，适合这里的清理和状态查询接口。

6.9 自定义消息 JoyRaw 与标准 Joy

本项目消息定义如下：

源码：`src/rc_handle_msgs/msg/JoyRaw.msg`，L1—L9。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_msgs/msg/JoyRaw.msg

```
1 # Unscaled joystick state, with the same axis/button indexes as jstest.
2 std_msgs/Header header
3
4 # Original signed 16-bit axis values from /dev/input/js*, without software
5 # deadzone, sign inversion, clipping, or normalization.
6 int16[] axes
7
8 # Unsigned 8-bit button values (normally 0: released, 1: pressed).
9 uint8[] buttons
```

字段	JoyRaw 的 C++ 常用类型	用途
`header.stamp`	`builtin_interfaces::msg::Time`	这次发布时设置的 ROS 时间
`header.frame_id`	`std::string`	逻辑坐标系或数据来源标识
`axes`	`std::vector<std::int16_t>`	原始轴数值
`buttons`	`std::vector<std::uint8_t>`	原始按键数值

消息字段由接口生成器产生，不需要手工创建 `joy_raw.hpp`。标准 `sensor_msgs/msg/Joy` 固定采用 `float32[] axes` 和 `int32[] buttons`，见 Humble 的 Joy.msg 原始定义。本项目只在发布 `/joy` 时转换字段存储类型。

IEEE 754 的 float32 可精确表示全部 int16 整数，因而 -32768 → -32768.0 不丢失整数值。数值可精确表示与是否归一化是不同问题：这个转换没有把轴缩小到 ±1。

6.10 消息包自己的构建文件

源码：`src/rc_handle_msgs/CMakeLists.txt`，L1—L14。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_msgs/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(rc_handle_msgs)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(std_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/JoyRaw.msg"
  DEPENDENCIES std_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

`rosidl_generate_interfaces` 根据 `msg/JoyRaw.msg` 生成 C++、Python 等支持的语言接口及通信类型支持。`DEPENDENCIES std_msgs` 来自消息中的 `std_msgs/Header`。生成头文件可以在构建产物中看到，不应手动修改它。

源码：`src/rc_handle_msgs/package.xml`，L1—L18。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_msgs/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>rc_handle_msgs</name>
  <version>0.1.0</version>
  <description>Raw joystick state with int16 axes and uint8 buttons.</description>
  <maintainer email="fyp@example.com">fyp</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>rosidl_default_generators</buildtool_depend>
  <depend>std_msgs</depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

`rosidl_default_generators` 是构建接口需要的工具；`rosidl_default_runtime` 是使用生成接口需要的运行依赖；`rosidl_interface_packages` 标记这是一个接口包。C++ 驱动依赖这个包，因此 `colcon` 会先生成消息，再编译驱动。

6.11 运算符和日志格式速查

写法	如何读	源码中的用途
`A && B`	A 和 B 都满足；有短路求值	检查已接收且向量未变
`A		B`	按位或	组合多个事件标志
`!condition`	逻辑取反	设备尚未打开
`flagsA	flagsB`	按位或	组合多个事件标志
`type & ~JS_EVENT_INIT`	按位清除初始化标志	识别真实事件种类
`condition ? A : B`	条件成立选 A，否则选 B	选择注入设备或真实设备
`T{...}`	花括号初始化	初始化状态、向量或时间对象
`nullptr`	空指针值	表示没有外部传入的设备
`function<T>(...)`	使用类型 T 实例化模板调用	声明参数、创建发布器

`RCLCPP_INFO`、`RCLCPP_WARN`、`RCLCPP_FATAL` 是日志宏，分别记录信息、警告和严重错误。当前项目使用 printf 风格的格式字符串：`%s` 对应 C 字符串，`%g` 对应浮点数，`%zu` 对应 size_t，`%d` 和 `%u` 分别用于相应有符号和无符号整数。

`std::string::c_str()` 提供以零字符结尾的字符串指针，供 `%s` 或 Linux API 使用；`std::array::data()` 返回连续缓冲区首地址，供 `read` 和 `ioctl` 写入。访问这些指针时，原字符串或容器对象必须仍然有效。

例如 `axes.assign(count, 0)` 使用“数量、初始值”版本；`joy.axes.assign(raw.axes.begin(), raw.axes.end())` 使用“起点、终点迭代器”版本，把一个容器区间复制并转换到另一个容器。它们同名但参数形式不同，属于函数重载。

07 joy_node_main.cpp：程序从哪里开始

文件路径为 `src/rc_handle_cpp/src/joy_node_main.cpp`。它提供可执行程序 `joy_node` 的入口，负责初始化、运行、异常出口和清理。Linux 设备解析与 ROS 消息组装在后面的类中实现。

7.1 完整源码

源码：`src/rc_handle_cpp/src/joy_node_main.cpp`，L1—L21。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/src/joy_node_main.cpp

```cpp
#include <exception>
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "rc_handle_cpp/joy_node.hpp"

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  int result = 0;
  try {
    rclcpp::spin(std::make_shared<rc_handle_cpp::JoyNode>());
  } catch (const std::exception & error) {
    RCLCPP_FATAL(rclcpp::get_logger("rc_handle_cpp"), "%s", error.what());
    result = 1;
  }
  if (rclcpp::ok()) {
    rclcpp::shutdown();
  }
  return result;
}
```

7.2 按执行顺序解释

L1—L5 包含异常、智能指针、ROS 和节点类声明。L7 的 `argc` 是命令行参数数量，`argv` 是参数字符串数组。它们被传给 L9 的 `rclcpp::init`，用于初始化 ROS 上下文并处理 ROS 命令行配置。

L10 把默认退出结果设成 0。L12 可以拆成两个教学步骤：

C++ 教学片段

```cpp
auto node = std::make_shared<rc_handle_cpp::JoyNode>();
rclcpp::spin(node);
```

第一步创建节点并执行构造函数，后者声明参数、创建发布器和定时器。第二步进入执行器处理过程，使定时器等回调有机会执行。`spin()` 期间通常持续运行，直到上下文关闭或出现未处理异常。

L13—L16 捕获创建节点或 spin 过程中传播出来的 `std::exception`，使用 `RCLCPP_FATAL` 打印信息，并设置退出结果 1。例如发布频率不合法，会在节点构造函数中抛出 `std::invalid_argument`，由这里记录。

L17—L19 检查 ROS 上下文是否仍有效，再调用 `rclcpp::shutdown()`。正常 `Ctrl+C` 触发关闭后，上下文可能已经失效，因此这里先判断。L20 返回退出码。注意 L9 的初始化位于 `try` 之外，当前 catch 不覆盖初始化本身抛出的异常。

7.3 main、类和可执行目标的关系

`JoyNode` 是 C++ 类名，`joy_node_main.cpp` 是源文件名，`joy_node` 是 CMake 目标及安装后的可执行名称，`rc_handle_cpp_node` 是默认 ROS 节点名。文件名中的 `_main` 不会自动成为运行命令。

终端命令

```
ros2 run rc_handle_cpp joy_node
```

这条命令先定位包前缀下的 `joy_node` 程序，由操作系统启动，再进入上面的 `main()`。launch 文件启动的也是这个可执行程序。

08 JoyNode：参数、定时器和消息发布

8.1 先读类声明 joy_node.hpp

源码：`src/rc_handle_cpp/include/rc_handle_cpp/joy_node.hpp`，L1—L41。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/include/rc_handle_cpp/joy_node.hpp

```
#ifndef RC_HANDLE_CPP__JOY_NODE_HPP_
#define RC_HANDLE_CPP__JOY_NODE_HPP_

#include <chrono>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "rc_handle_msgs/msg/joy_raw.hpp"
#include "sensor_msgs/msg/joy.hpp"
#include "rc_handle_cpp/joystick_device.hpp"

namespace rc_handle_cpp
{

class JoyNode : public rclcpp::Node
{
public:
  explicit JoyNode(
    const rclcpp::NodeOptions & options = rclcpp::NodeOptions(),
    std::unique_ptr<JoystickDevice> device = nullptr);

private:
  void tick();
  void publish_state(bool neutral = false);

  using SteadyClock = std::chrono::steady_clock;
  using SteadyTime = std::chrono::time_point<SteadyClock, std::chrono::duration<double>>;
  std::unique_ptr<JoystickDevice> device_;
  std::string frame_id_;
  std::string last_open_error_;
  std::chrono::duration<double> reconnect_interval_{1.0};
  SteadyTime next_open_{};
  rclcpp::Publisher<rc_handle_msgs::msg::JoyRaw>::SharedPtr raw_publisher_;
  rclcpp::Publisher<sensor_msgs::msg::Joy>::SharedPtr joy_publisher_;
  rclcpp::TimerBase::SharedPtr timer_;
};

}  // namespace rc_handle_cpp

#endif  // RC_HANDLE_CPP__JOY_NODE_HPP_
```

L16 表明类继承 `rclcpp::Node`。L19—L21 的构造函数有两个可选参数：`NodeOptions` 负责节点运行选项，设备智能指针允许注入自定义或模拟设备；默认空指针时创建真实设备。

L24 的 `tick()` 是每次定时器触发后的工作函数。L25 的 `publish_state(bool neutral = false)` 默认发布真实缓存；传入 true 时组装全零消息。默认值写在声明中，调用者可以直接写 `publish_state()`。

成员	类型或作用	为什么保存在类里
`device_`	独占设备对象	跨多个回调保存设备与状态

成员	类型或作用	为什么保存在类里
`frame_id_`	消息标识字符串	每次发布重复使用
`last_open_error_`	上次打开错误信息	避免重复刷同一条警告
`reconnect_interval_`	秒单位 duration	计算下次重连时间
`next_open_`	steady clock 时间点	决定本次回调能否重试打开
两个 publisher	发布器共享指针	保持 ROS 发布实体存活
`timer_`	定时器共享指针	保持周期回调有效

`using SteadyClock = ...` 是给类型起别名。`SteadyTime next_open_{}` 使用默认初始化的时间点；第一次 tick 时通常已经达到该时间点，因此会立即进行首次打开尝试。

8.2 完整实现 joy_node.cpp

源码：`src/rc_handle_cpp/src/joy_node.cpp`，L1—L102。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/src/joy_node.cpp

```cpp
#include "rc_handle_cpp/joy_node.hpp"

#include <cmath>
#include <stdexcept>
#include <system_error>
#include <utility>

#include "rcl_interfaces/msg/parameter_descriptor.hpp"

namespace rc_handle_cpp
{

JoyNode::JoyNode(
  const rclcpp::NodeOptions & options, std::unique_ptr<JoystickDevice> device)
: Node("rc_handle_cpp_node", options)
{
  rcl_interfaces::msg::ParameterDescriptor descriptor;
  descriptor.read_only = true;
  const auto path = declare_parameter<std::string>("dev", "/dev/input/js0", descriptor);
  const double rate = declare_parameter<double>("publish_rate", 50.0, descriptor);
  const double interval = declare_parameter<double>("reconnect_interval", 1.0, descriptor);
  frame_id_ = declare_parameter<std::string>("frame_id", "joy", descriptor);
  if (path.empty()) {
    throw std::invalid_argument("dev must not be empty");
  }
  if (!std::isfinite(rate) || rate < 1.0 || rate > 1000.0) {
    throw std::invalid_argument("publish_rate must be finite and in [1.0, 1000.0] Hz");
  }
  if (!std::isfinite(interval) || interval <= 0.0) {
    throw std::invalid_argument("reconnect_interval must be finite and > 0.0 seconds");
  }
  reconnect_interval_ = std::chrono::duration<double>(interval);
  device_ = device ? std::move(device) : std::make_unique<JoystickDevice>(path);
  raw_publisher_ = create_publisher<rc_handle_msgs::msg::JoyRaw>("joy_raw", 10);
  joy_publisher_ = create_publisher<sensor_msgs::msg::Joy>("joy", 10);
  // Wall timer and reconnect clock continue even if simulated ROS time pauses.
  timer_ = create_wall_timer(std::chrono::duration<double>(1.0 / rate), [this]() {tick();});
  RCLCPP_INFO(
```

src/rc_handle_cpp/src/joy_node.cpp（续）

```
    get_logger(), "Publishing raw int16/uint8 on %s and unscaled Joy on %s at %g Hz; device: %s",
    raw_publisher_->get_topic_name(), joy_publisher_->get_topic_name(), rate,
    device_->path().c_str());
}

void JoyNode::tick()
{
  if (!device_->is_open()) {
    if (SteadyClock::now() < next_open_) {
      return;
    }
    next_open_ = SteadyClock::now() + reconnect_interval_;
    try {
      device_->open();
    } catch (const std::system_error & error) {
      if (last_open_error_ != error.what()) {
        RCLCPP_WARN(
          get_logger(), "Cannot open %s: %s; retrying every %g s",
          device_->path().c_str(), error.what(), reconnect_interval_.count());
      }
      last_open_error_ = error.what();
      return;
    }
    last_open_error_.clear();
    RCLCPP_INFO(
      get_logger(), "Connected: %s; %zu axes, %zu buttons", device_->name().c_str(),
      device_->state().axes.size(), device_->state().buttons.size());
  }
  try {
    device_->read_state();
  } catch (const std::system_error & error) {
    RCLCPP_WARN(get_logger(), "Joystick disconnected/read failed: %s", error.what());
    device_->close();
    publish_state(true);
    next_open_ = SteadyClock::now() + reconnect_interval_;
    return;
  }
  publish_state();
}

void JoyNode::publish_state(bool neutral)
{
  rc_handle_msgs::msg::JoyRaw raw;
  raw.header.stamp = now();
  raw.header.frame_id = frame_id_;
  const auto & state = device_->state();
  if (neutral) {
    raw.axes.assign(state.axes.size(), 0);
    raw.buttons.assign(state.buttons.size(), 0);
  } else {
    raw.axes = state.axes;
    raw.buttons = state.buttons;
  }
  raw_publisher_->publish(raw);

  // Preserve the raw values on the fixed float32/int32 standard Joy schema.
  sensor_msgs::msg::Joy joy;
  joy.header = raw.header;
  joy.axes.assign(raw.axes.begin(), raw.axes.end());
  joy.buttons.assign(raw.buttons.begin(), raw.buttons.end());
  joy_publisher_->publish(joy);
}

}  // namespace rc_handle_cpp
```

8.3 L13—L42：构造函数完成什么

`: Node("rc_handle_cpp_node", options)` 调用基类构造函数，设置默认节点名，并让 ROS 运行选项作用于该节点。launch 或命令行可以再通过节点重命名规则覆盖运行时名称。

L17—L22 创建 `ParameterDescriptor`，并将 `read_only` 设为 true。四个参数的完整契约如下：

参数	声明类型	默认值	验证和单位
`dev`	`std::string`	`/dev/input/js0`	非空；设备路径
`publish_rate`	`double`	`50.0`	有限数；1.0—1000.0 Hz，包含端点
`reconnect_interval`	`double`	`1.0`	有限数且大于 0；秒
`frame_id`	`std::string`	`joy`	当前实现未额外要求非空

`declare_parameter<T>(name, default, descriptor)` 声明并取得参数的有效值；启动时传入的合法覆盖值会代替默认值。代码还检查 `std::isfinite`，避免 NaN、正负无穷等不适合构造周期的值。

L32 将秒数转换成 `std::chrono::duration<double>`。L33 使用三元表达式：如果外部传入设备对象，就移动到成员；否则用路径创建真实 `JoystickDevice`。这一步只创建 C++ 对象，首次打开设备要等到 `tick()`。

L34—L35 创建两个发布器，话题名是相对名 `joy_raw` 和 `joy`。数字 `10` 是 QoS 历史深度设置的简写，不是发布频率，也不是手柄按键数量。L37 的定时器周期为 `1.0 / rate` 秒；50 Hz 对应 0.02 秒，即 20 毫秒。

L37 中的 `[this]() {tick();}` 把回调绑定到当前对象。L38—L41 打印的是发布器解析后的实际话题名，因此进行重映射后日志可帮助你确认最终名称。

8.4 L44—L77：tick 的三个分支

分支一：设备尚未打开。 比较 steady clock 与 `next_open_`。如果还不到时间，直接 return；否则先安排下一次重试时间，再尝试 `device_->open()`。失败时保存错误文本并返回，不执行消息发布。

当连续打开失败且错误文本相同时，L54 的判断让警告不重复输出。如果错误发生变化，会记录新的警告。连接成功后清空历史错误，并打印设备维度。

分支二：设备已打开，但读取失败。 L68 调用 `read_state()`；捕获 `std::system_error` 后，先关闭设备，再调用 `publish_state(true)`，然后设置下次重连时刻并返回。下一次 tick 会重新进入“尚未打开”的分支。

分支三：读取成功。 L76 调用 `publish_state()`，将当前全部轴和按键发布。即便这次没有新设备事件，只要设备正常打开，仍会发布缓存状态。

重连间隔的判断依赖定时器回调实际被调度。因此“1 秒重试”表示达到下一次允许重试时刻后的回调进行尝试，不是具有硬实时精度的绝对承诺。

8.5 L79—L100：publish_state 怎样组装消息

L81 创建局部 `JoyRaw` 消息，L82 以节点的 `now()` 生成本次时间戳，L83 写入 frame_id。当前实现没有使用 Linux 事件中的 `time` 字段作为 ROS 时间戳。

L84 用只读引用取得设备缓存。如果 `neutral` 为 false，复制当前向量到消息；如果为 true，用相同元素数量创建全零向量。`close()` 不会清除缓存维度，因此断线时仍能组装“与断线前维度相同”的消息。这里清零的是要发布的消息，并不直接重写设备缓存。

L92 发布原始消息。L95—L99 创建标准 Joy，复制同一 header，再用向量区间赋值完成 int16→float32 和 uint8→int32 的元素转换。两个消息具有相同索引和同一发布批次的时间戳，但并不意味着网络上的两个话题会在同一瞬间到达订阅者。

8.6 为什么使用两种时钟

设备恢复属于真实进程运行中的工作。定时器采用 `create_wall_timer`，重连门限采用 `std::chrono::steady_clock`；Humble 的该定时器内部使用 steady clock，因此不会因为仿真 ROS 时间暂停就停止正常设备重试。

消息头时间戳使用 `now()`，它属于节点的 ROS 时钟语义。启用仿真时间后，这个值可能跟随 `/clock`。所以“重试仍在运行”与“消息时间戳变化”可以有不同表现。这个区分可以从已安装的 Humble `rclcpp/timer.hpp` 和本项目源文件共同验证。

09 JoystickDevice：Linux 手柄读取的底层

这一层不负责发布 ROS 消息，而是维护设备和输入状态。阅读时关注四个问题：如何打开设备、如何获得维度、如何把事件应用到状态、如何可靠关闭资源。

9.1 类和状态声明

源码：`src/rc_handle_cpp/include/rc_handle_cpp/joystick_device.hpp`，L1—L52。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/include/rc_handle_cpp/joystick_device.hpp

```
#ifndef RC_HANDLE_CPP__JOYSTICK_DEVICE_HPP_
#define RC_HANDLE_CPP__JOYSTICK_DEVICE_HPP_

#include <linux/joystick.h>

#include <cstddef>
#include <cstdint>
#include <string>
#include <vector>

namespace rc_handle_cpp
{

struct JoystickState
{
  std::vector<std::int16_t> axes;
  std::vector<std::uint8_t> buttons;

  void reset(std::size_t axis_count, std::size_t button_count);
  void apply(const js_event & event);
};

class JoystickDevice
{
public:
  explicit JoystickDevice(std::string path);
  virtual ~JoystickDevice();

  JoystickDevice(const JoystickDevice &) = delete;
  JoystickDevice & operator=(const JoystickDevice &) = delete;

  virtual void open();
  virtual void read_state();
  virtual void close() noexcept;
  virtual bool is_open() const noexcept;

  const JoystickState & state() const noexcept {return state_;}
  const std::string & path() const noexcept {return path_;}
  const std::string & name() const noexcept {return name_;}

protected:
  JoystickState state_;

private:
  std::string path_;
  std::string name_;
  int fd_{-1};
};

}  // namespace rc_handle_cpp

```

src/rc_handle_cpp/include/rc_handle_cpp/joystick_device.hpp（续）

```
52 #endif  // RC_HANDLE_CPP__JOYSTICK_DEVICE_HPP_
```

`JoystickState` 公开两个向量以及 `reset/apply`。`JoystickDevice` 公开设备生命周期接口；`state/path/name` 返回只读引用，生命周期由设备对象保证。调用者不要在设备对象销毁后继续使用这些引用。

`fd_{-1}` 使用 -1 表示未打开。打开成功时保存 Linux 返回的非负文件描述符。它是进程中的资源句柄，不是设备路径，也不是指针。`fd` 为 0 在 Linux 语义上也可以是合法描述符，因此检查写成 `>= 0`。

9.2 完整实现

源码：`src/rc_handle_cpp/src/joystick_device.cpp`，L1—L122。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/src/joystick_device.cpp

```
 1 #include "rc_handle_cpp/joystick_device.hpp"
 2
 3 #include <fcntl.h>
 4 #include <sys/ioctl.h>
 5 #include <unistd.h>
 6
 7 #include <array>
 8 #include <cerrno>
 9 #include <system_error>
10 #include <utility>
11
12 namespace rc_handle_cpp
13 {
14
15 static_assert(sizeof(js_event) == 8, "Unexpected Linux joystick event layout");
16
17 void JoystickState::reset(std::size_t axis_count, std::size_t button_count)
18 {
19   axes.assign(axis_count, 0);
20   buttons.assign(button_count, 0);
21 }
22
23 void JoystickState::apply(const js_event & event)
24 {
25   const auto type = event.type & ~JS_EVENT_INIT;
26   if (type == JS_EVENT_AXIS && event.number < axes.size()) {
27     axes[event.number] = event.value;
28   } else if (type == JS_EVENT_BUTTON && event.number < buttons.size()) {
29     if (event.value < 0 || event.value > 255) {
30       throw std::system_error(EIO, std::generic_category(), "button value does not fit uint8");
31     }
32     buttons[event.number] = static_cast<std::uint8_t>(event.value);
33   }
34 }
35
36 JoystickDevice::JoystickDevice(std::string path)
37 : path_(std::move(path)), name_(path_)
38 {
39 }
40
41 JoystickDevice::~JoystickDevice()
42 {
43   close();
44 }
45
46 void JoystickDevice::open()
47 {
```

src/rc_handle_cpp/src/joystick_device.cpp（续）

```cpp
  close();
  const int descriptor = ::open(path_.c_str(), O_RDONLY | O_NONBLOCK | O_CLOEXEC);
  if (descriptor < 0) {
    throw std::system_error(errno, std::generic_category(), "open " + path_);
  }
  try {
    std::uint8_t axis_count = 0;
    std::uint8_t button_count = 0;
    if (::ioctl(descriptor, JSIOCGAXES, &axis_count) < 0 ||
      ::ioctl(descriptor, JSIOCGBUTTONS, &button_count) < 0)
    {
      throw std::system_error(errno, std::generic_category(), "query joystick dimensions");
    }
    std::array<char, 128> name{};
    if (::ioctl(descriptor, JSIOCGNAME(name.size()), name.data()) >= 0) {
      name.back() = '\0';
      name_ = name.data();
    } else {
      name_ = path_;
    }
    state_.reset(axis_count, button_count);
  } catch (...) {
    ::close(descriptor);
    throw;
  }
  fd_ = descriptor;
}

void JoystickDevice::read_state()
{
  if (fd_ < 0) {
    throw std::system_error(EBADF, std::generic_category(), "joystick is not open");
  }
  // Bound work to 4096 events per timer tick so ROS callbacks remain responsive.
  std::array<js_event, 256> events{};
  for (int batch = 0; batch < 16; ++batch) {
    const auto bytes = ::read(fd_, events.data(), sizeof(events));
    if (bytes < 0) {
      const int error = errno;
      if (error == EAGAIN || error == EWOULDBLOCK) {
        return;
      }
      if (error == EINTR) {
        continue;
      }
      throw std::system_error(error, std::generic_category(), "read joystick");
    }
    if (bytes == 0) {
      throw std::system_error(ENODEV, std::generic_category(), "joystick disconnected");
    }
    if (bytes % sizeof(js_event) != 0) {
      throw std::system_error(EIO, std::generic_category(), "incomplete joystick event");
    }
    const auto count = static_cast<std::size_t>(bytes) / sizeof(js_event);
    for (std::size_t index = 0; index < count; ++index) {
      state_.apply(events[index]);
    }
  }
}

void JoystickDevice::close() noexcept
{
  if (fd_ >= 0) {
    const int descriptor = fd_;
    fd_ = -1;
    ::close(descriptor);
```

src/rc_handle_cpp/src/joystick_device.cpp（续）

```
114   }
115 }
116
117 bool JoystickDevice::is_open() const noexcept
118 {
119   return fd_ >= 0;
120 }
121
122 }  // namespace rc_handle_cpp
```

9.3 L15—L34：事件怎样变成完整状态

L15 的 `static_assert` 在编译阶段确认 `js_event` 大小为 8 字节。Linux 的该结构具有时间、值、类型、编号等字段；接口背景可对照 Linux joydev 编程接口。

L17—L21 的 `reset` 把向量重设为设备提供的维度并填零。设备每次成功打开时都会调用它，因此重连后不会直接沿用上一次设备的向量内容或长度。

L25 清除初始化标志位。例如初始化轴事件的 type 可以是 `0x82`，它包含 `0x80` 初始化标志和 `0x02` 轴类型。`event.type & ~JS_EVENT_INIT` 清除前者后得到轴类型，因此初始事件也能参与正常状态更新。

L26—L27 处理轴事件：编号合法时直接把 `event.value` 存入对应元素。没有除法、缩放、死区或符号反转。L28—L32 处理按键：首先检查值能否装入 uint8，再进行 `static_cast`。无效的负数或大于 255 的值抛出异常，而不是偷偷截断。

未知事件类型和越界编号被忽略。轴和按键分别编号，因此 `axes[0]` 与 `buttons[0]` 是两组独立序列中的第一个元素。

9.4 L36—L44：构造和析构

构造函数保存设备路径，并把路径作为默认设备名。它没有立即调用 Linux `open`。析构函数执行 `close()`，让正常退出或异常展开时也能释放设备资源。这种“资源随对象生命周期释放”的做法称为 RAII。

复制操作被禁止，避免复制文件描述符所有权。若两个独立对象都保存同一个 fd 并都 close，可能把资源错误关闭；当前类通过限制复制来消除这一类误用。

9.5 L46—L74：open 的资源管理顺序

先调用 `close()`，让反复打开行为有明确起点。L49 的 `::open` 用全局作用域符号指向 Linux 系统函数，区别于当前类的 `JoystickDevice::open()`。

标志	当前作用
`O_RDONLY`	只读打开手柄设备
`O_NONBLOCK`	无事件时读取立即返回相应错误码，避免卡住回调
`O_CLOEXEC`	进程执行新的程序映像时自动关闭此描述符

打开失败时，使用 `errno`、通用错误类别和上下文字符串构造 `std::system_error`。打开成功后先把 fd 保存在局部变量 `descriptor`，然后通过 `ioctl` 查询轴数、按键数和名称。

维度查询必须成功，否则当前设备不能按预期使用；名称查询失败则回退到路径，不必为缺少显示名称放弃整个设备。名称缓冲区末尾显式写入 `\0`，保证后续按 C 字符串解释时具有终止符。

初始化过程中如果发生任意异常，`catch (...)` 会关闭局部 fd，再用 `throw;` 原样重新抛出。只有一切准备完成后才将 `descriptor` 赋给 `fd_`。这能避免“设备打开成功，但查询失败导致 fd 泄漏”。

9.6 L76—L106：read_state 为何分批读取

每个缓冲区可容纳 256 个 `js_event`，每次 tick 最多尝试 16 批读取，因而最多应用 4096 个事件。上限用于限制一次回调的工作量，使执行器能继续处理退出等其他工作；它不是一个无限排空事件队列的循环。

`::read` 返回的是字节数，而不是事件数。完整事件数量由 `bytes / sizeof(js_event)` 计算。每个事件都交给 `state_.apply()`，同一个元素被多次更新后保留最后值。

read 的结果	代码处理	对当前节点的影响
正数字节数且事件完整	逐个应用事件，并可能继续读下一批	更新缓存状态
`EAGAIN` / `EWOULDBLOCK`	正常 return	当前没有待处理事件；仍可发布缓存
`EINTR`	继续下一次读取尝试	响应系统调用被中断的情况
其他负数错误	抛出 `system_error`	节点按读取失败进行恢复
0 字节	抛出断开设备错误	节点清零消息并重连
字节数不是事件大小的整数倍	抛出不完整事件错误	防止解析半条事件

先保存 `errno` 到局部 `error`，可以避免随后其他调用覆盖这个线程的错误状态。`bytes < 0` 在转换成 `size_t` 之前检查，避免负数变成巨大的无符号数。

9.7 L108—L120：close 与 is_open

`close()` 先确认 fd 非负，将成员设回 -1，再关闭保存的描述符。重复 close 不会重复操作已经关闭的 fd。函数标记 `noexcept`，适合析构清理路径。当前实现不检查 Linux `close` 的返回错误，不能把它描述成“会报告所有关闭故障”。

`is_open()` 只检查 fd 是否有效，并不主动探测手柄是否仍然物理连接。实际拔出通常要等读取失败才被节点发现。

9.8 “原始值”的准确含义

这里的原始值指 Linux `joydev` 接口返回的值，索引与同一设备的 `jstest` 相对应。应用层没有另加死区或标定；内核或设备自身可能已经做过映射或校准。因此“原样转发 joydev”不能推导成“直接拿到电位器 ADC 寄存器”。

10 joy_raw_listener.cpp：订阅者怎样写

10.1 完整源码

源码：`src/rc_handle_cpp/src/joy_raw_listener.cpp`，L1—L52。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/src/joy_raw_listener.cpp

```cpp
#include <memory>
#include <sstream>
#include <vector>

#include "rclcpp/rclcpp.hpp"
#include "rc_handle_msgs/msg/joy_raw.hpp"

class JoyRawListener : public rclcpp::Node
{
public:
  JoyRawListener()
  : Node("joy_raw_listener_cpp")
  {
    subscription_ = create_subscription<rc_handle_msgs::msg::JoyRaw>(
      "joy_raw", 10, [this](const rc_handle_msgs::msg::JoyRaw & msg) {
        if (received_ && msg.axes == axes_ && msg.buttons == buttons_) {
          return;
        }
        std::ostringstream output;
        output << "axes=[";
        for (std::size_t index = 0; index < msg.axes.size(); ++index) {
          output << (index == 0 ? "" : ", ") << msg.axes[index];
        }
        output << "] buttons=[";
        for (std::size_t index = 0; index < msg.buttons.size(); ++index) {
          // uint8_t streams as a character unless explicitly promoted.
          output << (index == 0 ? "" : ", ") << static_cast<unsigned int>(msg.buttons[index]);
        }
        output << "]";
        RCLCPP_INFO(get_logger(), "%s", output.str().c_str());
        axes_ = msg.axes;
        buttons_ = msg.buttons;
        received_ = true;
      });
  }

private:
  bool received_{false};
  std::vector<std::int16_t> axes_;
  std::vector<std::uint8_t> buttons_;
  rclcpp::Subscription<rc_handle_msgs::msg::JoyRaw>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<JoyRawListener>());
  if (rclcpp::ok()) {
    rclcpp::shutdown();
  }
  return 0;
}
```

10.2 创建订阅和注册回调

`JoyRawListener : public rclcpp::Node` 创建订阅节点。基类构造时指定名字 `joy_raw_listener_cpp`。L14—L15 创建 `JoyRaw` 类型的订阅器，订阅相对话题 `joy_raw`，历史深度为 10。

第三个参数是接收消息时调用的 Lambda。这里 `const JoyRaw & msg` 以只读引用访问本次消息。`subscription_` 保存在成员中，确保构造函数结束后订阅实体仍存在；只创建一个马上被销毁的局部订阅对象不能实现持续接收。

10.3 为什么手柄不动时不打印

L16 检查三件事：已经接收过消息、轴向量未变、按键向量未变。条件都成立就直接 return，省去重复日志。第一次消息即使全部为零，也会打印，因为 `received_` 初始为 false。

`msg.axes == axes_` 比较整个向量，包括长度和各元素；不是只比较地址。发布节点仍可以每秒发 50 条消息，但这个订阅示例可能很久只打印一行。

10.4 字符串流与 uint8 输出

L19 创建 `std::ostringstream`，用 `<<` 把各个数值拼成一段字符串。`index == 0 ? "" : ", "` 用于在第二个及以后的元素前加逗号。

`uint8_t` 在常见实现中是 unsigned char 类型，直接流输出可能当成字符。L27 的 `static_cast<unsigned int>` 让它以数字打印。转换只是为了显示，不会把消息中的字段类型改为 unsigned int。

L31—L33 保存本次向量和已接收标记，用作下次回调的比较基准。`output.str().c_str()` 在这一次完整日志调用表达式中有效；不要将这种临时字符串的指针保存起来，供表达式结束后继续使用。

10.5 在你自己的回调中取第一个轴

下面是可以替换或参考的教学片段；当前工程没有自动改成这个版本：

C++ 教学片段

```cpp
const std::int16_t axis0 = msg.axes.empty() ? 0 : msg.axes[0];
const std::uint8_t button0 = msg.buttons.empty() ? 0 : msg.buttons[0];
RCLCPP_INFO(
  get_logger(), "axis0=%d button0=%u",
  static_cast<int>(axis0), static_cast<unsigned int>(button0));
```

先检查 `empty()`，避免无轴或无按键设备导致越界。`axis0` 是第一个轴，不保证一定是左摇杆左右方向；具体索引需要结合设备输出观察。不要在不知道数值量级的情况下，直接把原始轴值当成机器人速度。

11 launch 和 YAML：把配置送到节点

11.1 launch 文件完整源码

源码：`src/rc_handle_cpp/launch/joy.launch.py`，L1—L29。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/launch/joy.launch.py

```
1  """Start the C++ raw joystick driver."""
2
3  from launch import LaunchDescription
4  from launch.actions import DeclareLaunchArgument
5  from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
6  from launch_ros.actions import Node
7  from launch_ros.substitutions import FindPackageShare
8
9
10 def generate_launch_description():
11     return LaunchDescription([
12         DeclareLaunchArgument(
13             'params_file',
14             default_value=PathJoinSubstitution([
15                 FindPackageShare('rc_handle_cpp'), 'config', 'joy.yaml']),
16             description='Path to the joystick parameter YAML file'),
17         DeclareLaunchArgument(
18             'joy_topic', default_value='joy', description='Standard Joy output topic'),
19         DeclareLaunchArgument(
20             'joy_raw_topic', default_value='joy_raw', description='Raw int16/uint8 output topic')
 >
21         Node(
22             package='rc_handle_cpp', executable='joy_node', name='rc_handle_cpp_node',
23             output='screen', parameters=[LaunchConfiguration('params_file')],
24             remappings=[
25                 ('joy', LaunchConfiguration('joy_topic')),
26                 ('joy_raw', LaunchConfiguration('joy_raw_topic')),
27             ],
28         ),
29     ])
```

虽然这是 C++ 节点，启动描述仍可用 Python 编写。`generate_launch_description()` 是该文件提供给 launch 系统的入口，返回由多个动作组成的 `LaunchDescription`。

11.2 L12—L20：声明启动参数

`DeclareLaunchArgument` 声明可从启动命令传入的参数，并提供默认值和说明。当前只显式声明三个启动参数：`params_file`、`joy_topic`、`joy_raw_topic`。

`params_file` 的默认值由 `FindPackageShare('rc_handle_cpp')` 找到包的安装 share 路径，再由 `PathJoinSubstitution` 拼上 `config/joy.yaml`。这使 launch 不必写死源码工作空间路径。

`LaunchConfiguration('params_file')` 表示运行 launch 时读取同名配置。它是延后求值的替换对象，而不是在导入 Python 文件时已经读出最终字符串。背景可参考 Humble 启动替换机制官方教程。

11.3 L21—L28：Node 动作启动什么

配置项	当前值	含义
package	rc_handle_cpp	在这个 ROS 包中查找程序
executable	joy_node	启动 CMake 安装的可执行入口
name	rc_handle_cpp_node	指定运行时节点名
output	screen	把进程输出显示在终端
parameters	启动参数指定的 YAML 路径	为节点加载参数覆盖值
remappings	两组源名和目标名	更改节点实际使用的话题名称

这里的 Node 来自 launch_ros.actions，表示一个启动动作。C++ 中继承的 rclcpp::Node 表示运行时 ROS 节点对象。它们共同参与启动过程，但属于不同语言和不同层次。

11.4 YAML 完整源码与作用域

源码：src/rc_handle_cpp/config/joy.yaml，L1—L6。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/config/joy.yaml

```yaml
/**:
  ros__parameters:
    dev: /dev/input/js0
    publish_rate: 50.0
    reconnect_interval: 1.0
    frame_id: joy
```

/** 是 ROS 参数文件中的节点匹配模式，让相应配置可匹配节点；ros__parameters 是参数内容的固定键。YAML 通过空格缩进表达层级，不要用 Tab 混排。publish_rate: 50.0 保持浮点写法，与 C++ 的 double 声明相符。

frame_id: joy 只改变消息头字段，不会改变话题名；dev 是 Linux 设备路径；publish_rate 同时决定周期性读取和发布的目标频率；reconnect_interval 是断线重试的时间门限，单位秒。

11.5 四种名称不要混在一起

类别	例子	由谁读取
launch 启动参数	joy_raw_topic、params_file	launch 替换系统
ROS 节点参数	publish_rate、dev	declare_parameter
ROS 话题名	/controller/joy_raw	发布器和订阅器
ROS 消息头内容	header.frame_id = joy	你的订阅业务逻辑

当前 launch 没有直接声明名为 `publish_rate` 的启动参数，所以不能仅凭写 `publish_rate:=100.0` 就假定它会覆盖节点参数。你可以传入自定义 YAML，或直接 `ros2 run ... --ros-args -p publish_rate:=100.0`。

11.6 自定义 YAML 和重映射实例

先复制一份配置用于学习，避免覆盖原始文件：

终端命令

```
cp /home/fyp/rc_handle/src/rc_handle_cpp/config/joy.yaml \
  /tmp/rc_handle_cpp_learning.yaml
```

编辑这份副本，例如将 `publish_rate` 设为 `100.0`、`reconnect_interval` 设为 `0.5`。然后使用副本启动：

终端命令

```
ros2 launch rc_handle_cpp joy.launch.py \
  params_file:=/tmp/rc_handle_cpp_learning.yaml \
  joy_raw_topic:=/controller/joy_raw \
  joy_topic:=/controller/joy
```

订阅示例需要对应修改自己的话题解析：

终端命令

```
ros2 run rc_handle_cpp joy_raw_listener --ros-args \
  -r joy_raw:=/controller/joy_raw
```

重映射只影响被配置的进程，不会自动修改另一台机器或另一个终端里所有订阅者。消息类型和消息字段内容也不会因为重映射而改变。

11.7 等价的直接运行配置

终端命令

```
ros2 run rc_handle_cpp joy_node --ros-args \
  --params-file /tmp/rc_handle_cpp_learning.yaml \
  -r __node:=rc_handle_cpp_node \
  -r joy_raw:=/controller/joy_raw \
  -r joy:=/controller/joy
```

`-r` 指定 ROS 名称重映射，`-p` 指定参数，`--params-file` 加载参数文件。`__node` 是节点重命名的特殊键。这些命令选项与 launch 的 Node 配置形成对应关系。

11.8 查看和检查参数

终端命令

```
ros2 launch rc_handle_cpp joy.launch.py --show-args
ros2 param list /rc_handle_cpp_node
ros2 param get /rc_handle_cpp_node publish_rate
ros2 param describe /rc_handle_cpp_node publish_rate
```

后三条需要驱动节点已经运行。当前四个自定义参数只读，因此尝试运行时设置通常会被拒绝。这是当前代码明确选择的参数契约；支持动态参数需要额外编写验证、定时器更新等逻辑。

12 把运行过程连成一条时间线

12.1 从 ros2 launch 到第一次消息

1. launch 查找 `rc_handle_cpp` 的安装目录，解析三个启动参数，加载 YAML。

2. launch 启动 `joy_node` 程序，把参数文件、节点名和话题重映射传入进程。

3. `main()` 初始化 ROS，创建 `JoyNode`。

4. 构造函数声明参数并校验，创建真实设备对象、两个发布器和定时器。

5. `spin()` 开始处理回调；首次 tick 尝试打开设备并读取初始化或待处理事件。

6. 状态更新成功后，组装 JoyRaw 和 Joy 两条消息。

7. 订阅者执行回调，并按“值发生变化才打印”的策略显示日志。

12.2 正常连接、掉线和恢复

阶段	设备层状态	节点行为	订阅侧可观察结果
启动时没有设备	未打开	定期尝试打开；相同错误日志去重	发布器可存在，但没有周期数据
成功打开	fd 有效，尺寸已查询	读取事件并按周期发完整状态	两个话题获得数据
暂时没有新事件	fd 有效；read 返回 EAGAIN	发布上次缓存状态	静止时也可测到周期频率
读取时检测到断线	关闭 fd，缓存维度保留	各发布一次全零状态，安排重连	接收者可能看到清零消息
等待重连	未打开	未到时刻则直接 return	没有持续数据流
重新打开成功	状态按新维度重置	恢复读取和发布	数组长度可能变化

断线全零表示“清除上一次输入”，不保证等于每种手柄扳机的物理静止数值。当前消息没有 `connected` 字段；如果下游要判断设备在线状态，可以结合消息接收超时进行设计。

12.3 发布周期与事件丢失的关系

假设 50 Hz 读取，一次回调间隔约 20 ms。某按键在两个回调之间先按下后松开，两条事件都可能在同一次读取中处理，最终发布状态是松开。当前结构保证完整状态转发，不保证把每一次短暂边沿都作为独立 ROS 消息保留下来。

提高 `publish_rate` 能缩短观察间隔，但不能把普通操作系统调度变为硬实时保证。如果业务要求完整边沿事件，需要设计独立事件消息或队列等机制，这已经超出当前实现。

12.4 QoS 深度、可靠性和持久性

本项目创建发布器和订阅器时传入 `10`，在 Humble 中对应 KeepLast 深度 10，并基于默认 QoS：Reliable、Volatile。已安装的 `rclcpp/qos.hpp` 给出了该简写构造方式。

深度 10 是通信实体缓存历史的限制，不是应用层永远保留的全部历史，也不是 10 Hz。Reliable 发布端可与请求 Best Effort 的订阅端匹配；测试中专门加入 `SensorDataQoS()` 订阅者验证这一组合。反向的可靠性组合不应直接类推。

Volatile 表示不为后来加入者保留历史样本。因此在设备断线并已经发完一次清零后才启动的订阅者，不应期待立刻补收那条历史清零消息。节点发现和消息可见性也需要一定时间。

12.5 当前实现的边界

当前程序以单设备、单线程 spin、周期状态发布为主。它不包含按钮业务映射、速度控制、归一化、死区、振动反馈、动态参数更新或永久输入记录。读懂这层驱动后，你可以在另一个订阅节点里添加应用层处理，使原始驱动契约保持清晰。

13 阅读测试：没有硬件怎样验证行为

13.1 两个测试文件测试什么

`test_joystick_device.cpp` 验证状态容器和设备失败路径；`test_joy_node.cpp` 注入 `ScriptedDevice`，通过真实 ROS 发布器与订阅器观察消息。它模拟设备可用性，但通信部分仍执行实际节点和消息交换过程。

测试组	Google Test 用例数	当前验证内容
`RawAxisTest` 参数化测试	7	-32768、-32767、-1、0、1、2048、32767 原值保持
`RawButtonTest` 参数化测试	4	0、1、2、255 保持为 uint8 可表示值
`JoystickState` 普通测试	3	状态保持与重设、忽略无效索引/类型、拒绝非法按键值
`JoystickDevice` 普通测试	1	缺失设备可重复尝试、重复关闭
`JoyNodeTest` 节点测试	2	DDS 消息类型和值、QoS、断线重连，以及非法参数拒绝
合计	17	15 个设备/状态用例 + 2 个节点用例

13.2 读一个最小状态测试

源码：`src/rc_handle_cpp/test/test_joystick_device.cpp`，L46—L59。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/test/test_joystick_device.cpp

```
46 TEST(JoystickState, HoldsStateUntilReleaseAndResetsOnReconnect)
47 {
48   rc_handle_cpp::JoystickState state;
49   state.reset(2, 2);
50   state.apply(event(1, JS_EVENT_BUTTON | JS_EVENT_INIT, 0));
51   state.apply(event(-1234, JS_EVENT_AXIS, 1));
52   EXPECT_EQ(state.buttons, (std::vector<std::uint8_t>{1, 0}));
53   EXPECT_EQ(state.axes, (std::vector<std::int16_t>{0, -1234}));
54   state.apply(event(0, JS_EVENT_BUTTON, 0));
55   EXPECT_EQ(state.buttons, (std::vector<std::uint8_t>{0, 0}));
56   state.reset(3, 1);
57   EXPECT_EQ(state.axes, (std::vector<std::int16_t>{0, 0, 0}));
58   EXPECT_EQ(state.buttons, (std::vector<std::uint8_t>{0}));
59 }
```

这个测试先创建状态并分配两个轴、两个按键，然后应用按键按下事件和轴变化事件。`EXPECT_EQ` 检查向量是否与预期相同。随后应用松开事件，最后以新的维度 reset，确认旧状态被重新初始化。

`TEST` 定义普通测试；`TEST_P` 定义参数化测试；`GetParam()` 获取当前参数。`ASSERT_TRUE` 失败时会结束当前测试函数，适合后续必须成立的前置条件；`EXPECT_*` 通常记录失败但继续执行后面的检查。

13.3 ScriptedDevice 为什么能替代真实设备

源码：`src/rc_handle_cpp/test/test_joy_node.cpp`，L28—L61。PDF 左侧为原始行号，长行可能折行；Markdown 保留原始代码。

src/rc_handle_cpp/test/test_joy_node.cpp

```
28 class ScriptedDevice : public rc_handle_cpp::JoystickDevice
29 {
30 public:
31   ScriptedDevice() : JoystickDevice("/scripted/joystick") {}
32
33   void open() override
34   {
35     ++open_attempts;
36     if (!available) {
37       throw std::system_error(ENOENT, std::generic_category(), "not connected");
38     }
39     state_.axes = next_axes;
40     state_.buttons = next_buttons;
41     connected = true;
42     ++connections;
43   }
44
45   void read_state() override
46   {
47     if (!available) {
48       throw std::system_error(ENODEV, std::generic_category(), "unplugged");
49     }
50   }
51
52   void close() noexcept override {connected = false;}
53   bool is_open() const noexcept override {return connected;}
54
55   bool available{false};
56   bool connected{false};
57   int open_attempts{0};
58   int connections{0};
59   std::vector<std::int16_t> next_axes{-32768, 32767, 1, -1};
60   std::vector<std::uint8_t> next_buttons{1, 255};
61 };
```

这个类继承 `JoystickDevice` 并覆盖虚函数。`available` 控制设备是否存在，`connected` 控制打开状态，`next_axes/next_buttons` 提供希望节点读到的数值。测试把它装进 `unique_ptr`，移动到 `JoyNode` 构造函数中，因此节点仍调用同一套接口。

测试保存的裸指针 `scripted` 只是观察和控制已经由节点拥有的对象，不负责释放；其有效范围受节点及其设备对象生命周期约束。这样的依赖注入让“断线—重连—维度变化”可以重复、快速地测试。

13.4 节点测试中的执行器和等待

节点测试创建独立 Context 和唯一命名空间，减少不同测试之间的干扰。然后把驱动与订阅节点加入 `SingleThreadedExecutor`，反复 `spin_some()`，直到观察到预期消息或到达超时。

它首先验证缺设备时没有消息，然后将设备设为可用，检查原始字段、标准 Joy 转换和 header；之后模拟拔出，检查清零与停止发布；最后设置新的数组维度并恢复设备，检查重连结果。

非法参数测试覆盖频率 0、1001、NaN、无穷，重连间隔 0、负数、NaN、无穷，以及空设备路径。读到这些测试时，应把它们看作可执行的接口契约。

13.5 本次实际执行的检查

终端命令

```
source /opt/ros/humble/setup.bash
cd /home/fyp/rc_handle
colcon build --symlink-install --packages-up-to rc_handle_cpp
source install/setup.bash
colcon test --packages-select rc_handle_cpp --event-handlers console_direct+
colcon test-result --test-result-base build/rc_handle_cpp --verbose
```

本次检查：构建成功，两个 CTest 测试入口通过，其中包含 17 个 Google Test 用例；colcon 聚合报告显示 `19 tests, 0 errors, 0 failures, 0 skipped`。这个 19 是汇总工具统计包含的测试条目，不应与 17 个 Google Test 用例混成一个口径。

另外已实际执行 `ros2 pkg executables rc_handle_cpp`、`ros2 interface show rc_handle_msgs/msg/JoyRaw` 和 `ros2 launch rc_handle_cpp joy.launch.py --show-args`，结果与教程描述相符。本次 `/dev/input/js0` 不存在，因此没有验证真实硬件的轴数量、按键布局或现场频率。

14 动手练习：每一步都能检查结果

以下练习是学习建议，文档生成过程没有自动替你修改驱动实现。先完成前四项，再尝试增加新程序。

14.1 练习一：查清名称关系

在终端分别运行 `ros2 pkg executables rc_handle_cpp` 和 `ros2 node list`。目标是区分包名、程序名和运行时节点名。停止程序后再运行 `node list`，观察节点消失，而安装好的程序入口仍存在。

判断题：`ros2 run rc_handle_cpp rc_handle_cpp_node` 能否直接运行？在当前安装规则中，这不是可执行名称，应使用 `joy_node`。节点名由程序创建，不能代替程序名。

14.2 练习二：用命令行改变频率

终端命令

```
ros2 run rc_handle_cpp joy_node --ros-args -p publish_rate:=100.0
```

节点正常启动后，使用 `ros2 param get` 检查有效值。有可用手柄时，再用 `ros2 topic hz` 观察目标频率变化。然后停止程序，尝试 `-p publish_rate:=0.0`，观察错误日志和失败退出。

你应能在源码 L26—L27 找到失败原因，并解释为什么缺失设备会重试，而非法频率会阻止节点启动：前者是运行过程中可恢复的设备故障，后者在构造节点时违反配置要求。

14.3 练习三：重映射后保持订阅匹配

将发布话题改为 `/study/joy_raw`，先保持订阅示例默认不变，观察收不到数据的现象。然后给订阅示例增加 `-r joy_raw:=/study/joy_raw`，观察恢复。通过 `ros2 topic list -t` 和 `ros2 node info` 解释这个过程。

完成标准是能指出发布和订阅的最终解析名称，而不是仅根据源代码中的字符串猜测。

14.4 练习四：在读消息前检查长度

在订阅回调中打印 `axes.size()` 和 `buttons.size()`。如果要访问 `axes[5]`，先确认 `size() > 5`。试着说明为什么“这次手柄有八个轴”不能成为所有设备的编译时假定。

14.5 练习五：增加一个最小订阅程序

教学示例：新增 `src/axis0_listener.cpp`，内容如下。这个程序只在收到消息时打印第一个轴，真实输入可能让日志较多。

C++ 教学片段

```cpp
#include <cstdint>
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "rc_handle_msgs/msg/joy_raw.hpp"

class Axis0Listener : public rclcpp::Node
{
public:
  Axis0Listener() : Node("axis0_listener")
```

C++ 教学片段（续）

```cpp
  {
    sub_ = create_subscription<rc_handle_msgs::msg::JoyRaw>(
      "joy_raw", 10, [this](const rc_handle_msgs::msg::JoyRaw & msg) {
        if (msg.axes.empty()) {
          return;
        }
        RCLCPP_INFO(get_logger(), "axis0=%d", static_cast<int>(msg.axes[0]));
      });
  }

private:
  rclcpp::Subscription<rc_handle_msgs::msg::JoyRaw>::SharedPtr sub_;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<Axis0Listener>());
  if (rclcpp::ok()) {
    rclcpp::shutdown();
  }
  return 0;
}
```

在 `CMakeLists.txt` 的 `ament_package()` 之前加入这组规则，建议放在现有两个可执行目标附近：

CMake 片段

```cmake
add_executable(axis0_listener src/axis0_listener.cpp)
ament_target_dependencies(axis0_listener rclcpp rc_handle_msgs)
install(TARGETS axis0_listener DESTINATION lib/${PROJECT_NAME})
```

原包已经声明了 `rclcpp` 和 `rc_handle_msgs`，所以这次练习不需要重复添加相同 XML 依赖。如果你把程序放到一个新包，则应在新包的 XML 和 CMake 中分别声明和使用依赖。

终端命令

```
colcon build --symlink-install --packages-up-to rc_handle_cpp
source install/setup.bash
ros2 run rc_handle_cpp axis0_listener
```

当前项目没有预先包含 `axis0_listener` 目标；只有完成上述新增和构建后，该命令才有效。

14.6 练习六：理解归一化应放在哪里

如果下游需要 `[-1, 1]` 输入，可以在自己新增的订阅节点中显式处理。例如下面的教学公式将 int16 的两个极值分别映射到 -1 和 1：

C++ 教学片段

```cpp
const auto value = msg.axes[0];  // Check array length before this line.
const double normalized = value < 0
  ? static_cast<double>(value) / 32768.0
  : static_cast<double>(value) / 32767.0;
```

这只是一个数值映射示例，不包含设备标定、死区或业务控制策略。如果要发布结果，建议新建清楚命名的话题并说明消息契约；不要让已有原始话题悄悄改成另一种量级。

14.7 建议的七次学习安排

次序	学习内容	可以自检的成果
第 1 次	工作空间、构建、三个终端	能独立启动并查询话题
第 2 次	XML 依赖、CMake 目标与安装	能解释程序放进 src 后为何还不能运行
第 3 次	main、继承、智能指针、回调	能追踪程序从入口到首次 tick
第 4 次	Linux 事件、状态容器、异常	能解释 EAGAIN 与断线的区别
第 5 次	参数、YAML、launch、重映射	能更改配置而无需修改 C++ 常量
第 6 次	新增订阅程序	能同时修改源码和构建规则
第 7 次	测试、时钟、边界情况	能说明未插设备、短按、重连时的行为

15 常见问题与定位方法

15.1 Package not found

先确认拼写是 `rc_handle_cpp`，检查目标包是否构建成功，再在当前终端加载安装环境。使用 `ros2 pkg prefix rc_handle_cpp` 检查 ROS 实际找到哪个安装前缀；存在多个工作空间时，这能帮助发现加载了旧版本。

终端命令

```
source /opt/ros/humble/setup.bash
source /home/fyp/rc_handle/install/setup.bash
ros2 pkg prefix rc_handle_cpp
```

15.2 找不到 rc_handle_msgs 或 joy_raw.hpp

包配置找不到时，检查消息包是否在工作空间、XML 是否有依赖、是否使用 `--packages-up-to rc_handle_cpp` 构建。头文件找不到时，还要检查拼写是 `rc_handle_msgs/msg/joy_raw.hpp`，以及目标是否通过 `ament_target_dependencies` 使用了消息包。

不要在源码目录中手动创建一个同名头文件来掩盖接口生成失败；应修正消息包构建和依赖。

15.3 undefined reference

这是链接阶段常见错误。对于 `JoyNode` 或 `JoystickDevice` 方法，检查实现 `.cpp` 是否进入驱动库，以及 `joy_node` 是否链接该库。只在头文件里声明函数，却没有定义或没有链接定义所在目标，都可能触发此类错误。

15.4 ros2 run 找不到可执行程序

确认运行名是 `joy_node` 或 `joy_raw_listener`。新增程序时检查 `add_executable`、依赖和 `install(TARGETS ... DESTINATION lib/${PROJECT_NAME})` 三部分，然后重新构建和 source。

终端命令

```
ros2 pkg executables rc_handle_cpp
```

如果能在 `build/` 看到二进制，但 ROS 列不出入口，优先检查安装和当前环境，而不是立即改 C++。

15.5 启动了但没有消息

按顺序观察：节点是否存在、发布器是否存在、设备是否打开成功、话题是否被重映射、订阅消息类型和 QoS 是否匹配。当前驱动“没有设备时保持运行”是预期行为，不表示一定正在发布。

终端命令

```
ros2 node list
ros2 topic info /joy_raw --verbose
ls -l /dev/input/js*
ls -l /dev/input/by-id/*-joystick
```

`ros2 topic echo` 只是订阅工具，不会帮你启动驱动。显式写消息类型能让工具知道如何订阅，也不能制造一个实际不存在的发布进程。

15.6 Permission denied 或路径不存在

先用 `ls -l` 检查设备节点的属主、组和权限，用 `id` 检查当前用户组。对于系统规则将设备分配给 input 组的环境，管理员可按实际策略授权用户并重新登录；也可设置恰当的 udev 规则。不要把修改权限和设备不存在混为一件事。

若 `/dev/input/js0` 根本不存在，先解决 Linux 是否识别手柄及是否提供 joydev 节点的问题。虚拟机或 WSL 环境还需检查设备是否传入该 Linux 环境。更改 ROS 话题名不会修复 Linux 设备缺失。

设备编号可能变化，`/dev/input/by-id/*-joystick` 中的稳定路径在存在时可作为参数 `dev`。实际路径应以本机列出的结果为准。

15.7 参数设置不生效或类型不匹配

先用 `ros2 param get` 确认有效值。检查 YAML 是否由这次 `params_file` 指向、缩进是否正确、节点匹配模式是否适用。当前 double 参数建议使用 `50.0`、`1.0` 等写法，避免把整数参数类型误当成浮点。

如果已经启动后运行 `ros2 param set`，当前只读描述符会拒绝修改。应修改启动配置并重启，或者自行扩展动态参数实现。改了 C++ 默认值却一直加载旧 YAML 覆盖值，也不会看到默认值变化。

15.8 日志很少，但发布频率正常

订阅示例按向量是否变化去重打印。测频率应使用 `ros2 topic hz`，不能数订阅示例日志。打开失败警告也会按错误文本去重，因此没有重复警告不代表节点已经放弃重试。

15.9 轴值不是 -1 到 1

这是本项目的预期契约，`/joy_raw` 和 `/joy` 都保留原始量级。读取标准 `Joy` 并不能自动推断上游已经执行归一化。请先查看数组实际值，再决定是否在应用层缩放。

15.10 两个发布器同时存在

使用 `ros2 topic info /joy_raw --verbose` 看发布者。若你同时启动 Python 版、C++ 版或重复启动同一版，可能有多个发布者发送同一设备数据。停止多余进程，再进行一对一观察。

15.11 本地节点能运行，另一个终端或设备看不到

先确认两个终端加载相同消息接口，且 `ROS_DOMAIN_ID` 等通信环境兼容，再检查最终话题名和 QoS。本教程的验证范围是本机环境；跨机器发现还与网络、DDS 配置等相关，不能仅靠改源文件里的 topic 字符串解决。

16 速查表与自测题

16.1 文件职责速查

文件	一句话职责	关键入口
package.xml	描述包身份、依赖和构建类型	<package>
CMakeLists.txt	创建库/程序，配置安装导出	add_library、add_executable
joy_node_main.cpp	程序入口、执行器、异常出口	main
joy_node.hpp	声明 ROS 节点的接口和成员	class JoyNode
joy_node.cpp	参数、发布、周期读取与重连	tick、publish_state
joystick_device.hpp	声明设备和状态接口	两个类型声明
joystick_device.cpp	解析 Linux 手柄事件并管理 fd	open、read_state、apply
joy_raw_listener.cpp	订阅并打印变化	消息回调
joy.launch.py	组织启动参数、节点和重映射	generate_launch_description
joy.yaml	设置节点启动参数	ros__parameters
JoyRaw.msg	定义原始数值消息字段	axes、buttons

16.2 常用命令速查

终端命令

```
# Build and load the workspace.
colcon build --symlink-install --packages-up-to rc_handle_cpp
source install/setup.bash

# Launch or run: choose one publisher command.
ros2 launch rc_handle_cpp joy.launch.py
ros2 run rc_handle_cpp joy_node

# Inspect graph and data from another sourced terminal.
ros2 node list
ros2 topic list -t
ros2 topic info /joy_raw --verbose
ros2 topic echo /joy_raw rc_handle_msgs/msg/JoyRaw
ros2 topic hz /joy_raw
ros2 interface show rc_handle_msgs/msg/JoyRaw

# Inspect configuration and test results.
ros2 launch rc_handle_cpp joy.launch.py --show-args
ros2 param get /rc_handle_cpp_node publish_rate
colcon test --packages-select rc_handle_cpp
colcon test-result --test-result-base build/rc_handle_cpp --verbose
```

上面的运行、echo 和 hz 命令会持续占用终端；按 `Ctrl+C` 结束当前命令后再输入下一条，或分别打开多个终端。构建和加载命令需要在工作空间根目录执行。

16.3 中英文术语

英文	中文理解	本项目实例
workspace / package	工作空间 / 包	根目录 / `rc_handle_cpp`
executable / target	可执行程序 / 构建目标	`joy_node` / 驱动库目标
node / topic / message	节点 / 话题 / 消息	`JoyNode` / `joy_raw` / `JoyRaw`
publisher / subscription	发布器 / 订阅实体	两个 publisher / `subscription_`
callback / executor	回调 / 执行器	`tick` / `spin`
declaration / definition	声明 / 定义	`.hpp` 接口 / `.cpp` 实现
compile / link / install	编译 / 链接 / 安装	源码到目标文件，再组成程序并部署
dependency / export	依赖 / 导出	消息包依赖 / 下游可用的库目标
namespace / remapping	命名空间 / 重映射	`/robot1` / `joy_raw:=...`
parameter / launch argument	节点参数 / 启动参数	`publish_rate` / `params_file`
nonblocking / file descriptor	非阻塞 / 文件描述符	`O_NONBLOCK` / `fd_`
steady clock / duration	单调时钟 / 时间间隔	重连计时 / `reconnect_interval_`
reconnect interval	两次重连尝试之间的等待间隔	默认 1.0 秒
raw / normalized	原始量级 / 归一化量级	当前原值 / 练习中的缩放值

16.4 十道理解题及参考答案

1. 为什么声明了 XML 依赖还要设置 CMake 依赖？ XML 为包工具描述需求，CMake 为具体编译目标建立头文件和链接关系。

2. 为什么 `joy_node_main.cpp` 不直接读取手柄？ 入口承担启动职责，设备和节点类承担具体行为，便于复用和测试。

3. `create_publisher<JoyRaw>("joy_raw", 10)` 中的 10 是什么？ QoS 历史深度，不是频率。

4. 没有新按键事件时，正常连接的节点还会发布吗？ 会，周期性发布缓存中的完整状态。

5. 断线后会一直发布全零消息吗？ 不会。检测到读取故障时各发一次，然后等待重连。

6. `uint8_t` 为何要转换后再打印？ 常见实现会按字符流输出它；转为 unsigned int 可明确输出数字。

7. 为什么 `std::move(device)` 后不应继续假定原 unique_ptr 拥有设备？ 所有权已转移给节点；原 unique_ptr 被移动后不再拥有该对象。

8. 修改 `.cpp` 后只 source 是否足够？ 不足够；必须重新构建生成新的机器码，再启动新进程。

9. 改变 `frame_id` 会改变话题名吗？ 不会。它修改消息头内容，话题名称由发布器解析和重映射决定。

10. 为什么同一次读取中可能看不到一个很短的按键脉冲？ 按下和松开事件都应用到同一状态缓存，周期结束时只发布最终状态。

17 参考资料、版本和源码清单

17.1 本地源码是解释当前行为的依据

本书的完整源码清单直接从 `src/rc_handle_cpp` 与 `src/rc_handle_msgs` 摘录，未为生成教程而修改驱动实现。文件行号按本次快照排列；将来项目改动后，应以更新后的源码为准。

源文件	行数	SHA-256 前 16 位
src/rc_handle_cpp/package.xml	21	4f9ef73a20275f59
src/rc_handle_cpp/CMakeLists.txt	57	245ec197af98f517
src/rc_handle_msgs/msg/JoyRaw.msg	9	018c9db0325eb580
src/rc_handle_msgs/CMakeLists.txt	14	2c99b2f253d3184a
src/rc_handle_msgs/package.xml	18	d77493c778713a37
src/rc_handle_cpp/src/joy_node_main.cpp	21	f950764408983bfe
src/rc_handle_cpp/include/rc_handle_cpp/joy_node.hpp	41	3b5c215c12e09098
src/rc_handle_cpp/src/joy_node.cpp	102	a8b994606720262f
src/rc_handle_cpp/include/rc_handle_cpp/joystick_device.hpp	52	2a652027833c347d
src/rc_handle_cpp/src/joystick_device.cpp	122	82a3cfaa52b41950
src/rc_handle_cpp/src/joy_raw_listener.cpp	52	bba59b22fcb2d31c
src/rc_handle_cpp/launch/joy.launch.py	29	3d8b0a858bccba75
src/rc_handle_cpp/config/joy.yaml	6	614d74615998443b
src/rc_handle_cpp/test/test_joystick_device.cpp	94	d54760f600d85b68
src/rc_handle_cpp/test/test_joy_node.cpp	195	affe92d4201e8a97

17.2 官方资料入口

下列资料帮助进一步学习通用接口。正文中相应概念也提供了直接链接。ROS 文档网站可能要求浏览器验证，遇到这种情况可查看这里给出的官方仓库源码版本。

- ROS 2 Humble 的 ament_cmake 用户说明：包构建、目标、安装和导出。
- REP-149 包清单格式 3：XML 标签语义和依赖类别。
- ROS 2 Humble 的 launch substitutions 教程：启动替换、延后求值和参数传递。
- sensor_msgs/msg/Joy 的 Humble 定义：标准 Joy 字段类型。

- Linux joydev 编程接口：设备事件与非阻塞读取背景。

本次还核对了本机 Humble 安装中的 `rclcpp/qos.hpp`、`rclcpp/timer.hpp`、`launch/actions/declare_launch_argument.py` 和 `launch/substitutions/launch_configuration.py`，用于确认 QoS 简写、定时器时钟和启动参数读取行为。

17.3 继续阅读源码时的检查顺序

先读类型声明，找到对象保存的状态；再找构造函数，看资源如何创建；然后顺着主回调看正常路径；最后追踪异常、退出和重连路径。每遇到一个新依赖，回到 XML 和 CMake 检查它如何被声明、查找和应用到目标。这套顺序也适用于其他 ROS 2 C++ 小型驱动包。